RV College of Engineering®
Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

Go, change the world

Federated Heterogeneous Graph Neural Networks for TBML Detection

A Major Project Report (CS481P)

Submitted by,

Nikhil Vasu	**1RV22CS128**
Rohith Biradar	**1RV22CS164**
Suraj Chanaveeragoudra	**1RV22CS211**

Under the guidance of

Dr. Nagaraj G. S
Professor
Dept. of CSE
RV College of Engineering

Prof. Sanjana Ravindra Otihal
Assistant Professor
Dept. of CSE
RV College of Engineering

In partial fulfillment of the requirements for the degree of Bachelor of Engineering in

Computer Science and Engineering

2025-26

RV College of Engineering®, Bengaluru

(*Autonomous institution affiliated to VTU, Belagavi*)

Department of Computer Science and Engineering

CERTIFICATE

Certified that the major project (CS481P) work titled ***Federated Heterogeneous Graph Neural Networks for TBML Detection*** is carried out by **Nikhil Vasu** (**1RV22CS128**), **Rohith Biradar** (**1RV22CS164**) and **Suraj Chanaveeragoudra** (**1RV22CS211**) who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the requirements for the degree of **Bachelor of Engineering** in **Computer Science and Engineering** of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project report deposited in the departmental library. The major project report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said degree.

Guide	Head of the Department	Principal
Dr. Nagaraj G. S	Dr. Shanta Rangaswamy	Dr. K. N. Subramanya

External Viva

Name of Examiners	Signature with Date
1.	
2.	

DECLARATION

We, **Nikhil Vasu**, **Rohith Biradar** and **Suraj Chanaveeragoudra** students of eighth semester B.E., Department of Computer Science and Engineering, RV College of Engineering, Bengaluru, hereby declare that the major project titled '**Federated Heterogeneous Graph Neural Networks for TBML Detection**' has been carried out by us and submitted in partial fulfilment for the award of degree of **Bachelor of Engineering** in **Computer Science and Engineering** during the year 2025-26.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and We will be one of the authors of the same.

Place: Bengaluru

Date:

Name Signature

1. Nikhil Vasu(1RV22CS128)
2. Rohith Biradar(1RV22CS164)
3. Suraj Chanaveeragoudra(1RV22CS211)

ACKNOWLEDGEMENTS

We are indebted to our guide, **Dr. Nagaraj G. S**, Professor, Department of CSE, RVCE for the wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

We also express our gratitude to our panel members **Dr. Nagaraj G. S**, Professor, Department of CSE, RVCE and **Prof. Sanjana Ravindra Otihal**, Assistant Professor, Department of CSE, RVCE for the valuable comments and suggestions during the phase evaluations.

Our sincere thanks to the project coordinators **Dr. Chethana R Murthy**, Professor, Department of CSE and **Prof. Deepika Dash**, Assistant Professor, for their timely instructions and support in coordinating the project.

Our gratitude to **Prof. Narashimaraja P**, Department of ECE, RVCE for the organized latex template which made report writing easy and interesting.

Our sincere thanks to **Dr. Shanta Rangaswamy**, Professor and Head, Department of Computer Science and Engineering, RVCE for the support and encouragement.

We express sincere gratitude to our beloved Professor and Vice Principal, **Dr. Geetha K S**, RVCE and Principal, **Dr. K. N. Subramanya**, RVCE for the appreciation towards this project work.

We thank all the teaching staff and technical staff of Computer Science and Engineering department, RVCE for their help.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

ABSTRACT

Trade-Based Money Laundering (TBML) has become a major challenge in modern financial systems due to the increasing complexity and scale of global trade transactions. Conventional anti-money laundering systems primarily rely on rule-based monitoring and traditional machine learning techniques, which are often inadequate for identifying hidden transactional dependencies and evolving laundering strategies. These approaches suffer from limitations such as high false positive rates, poor relational learning capability, and reduced scalability in large financial networks. To address these issues, this report presents a Federated Heterogeneous Graph Neural Network (HGNN) framework for real-time TBML detection capable of modeling complex trade relationships while preserving institutional data privacy.

The primary objective of this work is to develop a scalable and privacy-preserving TBML detection architecture using graph-based deep learning methodologies. The proposed framework represents financial ecosystems as heterogeneous graphs consisting of entities such as accounts, importers, exporters, transactions, and trade documents. Graph Attention Network (GAT)-based convolutional layers are employed to learn structural and semantic relationships between interconnected entities for effective anomaly detection. Federated learning mechanisms are incorporated to facilitate collaborative model training across distributed financial institutions without centralized data sharing. The computational framework includes graph construction, feature extraction, node embedding generation, suspicious activity classification, and real-time risk score prediction.

The proposed system is implemented using Python, PyTorch Geometric, FastAPI, Memgraph, PostgreSQL, and React.js. Experimental evaluation is carried out on simulated TBML datasets containing both legitimate and suspicious trade transactions. Performance analysis is conducted using standard classification metrics including accuracy, precision, recall, and F1-score. The experimental results indicate that the proposed framework effectively captures hidden laundering patterns and improves fraud detection capability when compared with conventional machine learning approaches. The architecture further supports scalable deployment, reduced false positive rates, and efficient real-time monitoring of high-volume trade transactions.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1

Introduction to Trade-Based Money Laundering

CHAPTER 1

INTRODUCTION TO TRADE-BASED MONEY LAUNDERING

The rapid growth of international trade and digital financial infrastructure has significantly increased the complexity of monitoring global financial transactions. Alongside economic globalization, financial institutions are facing increasingly sophisticated forms of illicit financial activities that exploit weaknesses in conventional Anti-Money Laundering (AML) systems. Among these, Trade-Based Money Laundering (TBML) has emerged as one of the most challenging financial crimes due to its ability to conceal illegal fund transfers within legitimate trade operations. The distributed and interconnected nature of modern trade ecosystems necessitates intelligent, scalable, and privacy-preserving analytical frameworks capable of identifying hidden transactional relationships in real time.

> *"For financial institutions, TBML is uniquely challenging because illicit funds are embedded within legitimate trade transactions, leaving traditional AML controls blind to the risks."*
>
> *– Dr. Sebastian Hetzler, IMTF Co-CEO [1]*

As highlighted in the above statement, Trade-Based Money Laundering exploits the complexity of international commerce by embedding illicit financial flows within apparently legitimate trade transactions. This significantly limits the effectiveness of conventional compliance systems that primarily rely on isolated transaction monitoring and static rule-based verification mechanisms.

1.1 Overview of Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) is defined as the process of disguising the proceeds of crime and transferring value through legitimate international trade transactions in an attempt to legitimize illicit financial activities [2]. Unlike conventional money laundering mechanisms that rely on direct movement of funds, TBML conceals illegal financial flows within normal commercial trade operations such as import-export activities, invoice settlements, and cross-border transactions. Due to the involvement of multiple intermediaries, financial institutions, customs agencies, and international jurisdictions,

identifying suspicious trade behavior becomes significantly difficult for conventional Anti-Money Laundering systems.

1.1.1 Global Financial Crime Scenario

The increasing volume of international trade and cross-border financial transactions has considerably complicated the process of monitoring illicit financial activities. According to the United Nations Office on Drugs and Crime (UNODC), money laundering activities account for nearly 2% to 5% of the global Gross Domestic Product (GDP), corresponding to approximately $800 billion to $2 trillion annually [3]. Despite the deployment of large-scale compliance frameworks and transaction monitoring systems, Europol reports that global authorities successfully intercept less than 1% of illicit financial flows [4]. These statistics indicate the growing sophistication of modern laundering networks and the limitations of existing monitoring infrastructures.

The large-scale movement of goods, services, and financial assets across international markets creates an environment where illicit transactions can be concealed within legitimate commercial activities. As financial institutions continue to strengthen Know-Your-Customer (KYC) regulations and banking surveillance systems, criminal organizations increasingly exploit trade ecosystems to bypass conventional financial monitoring mechanisms.

Table 1.1: Key Statistics Related to Global Trade-Based Financial Crime

Financial Crime Metric	**Estimated Value**
Global GDP associated with money laundering annually	2% – 5%
Estimated annual TBML economic volume	$1.6T – $2.0T
Illicit flows linked to trade misinvoicing in developing economies	87%
Global interception rate of illicit financial flows	<1%
Reduction in false positive alerts using AI-driven monitoring systems	Up to 40%

1.1.2 Trade-Based Money Laundering

Trade-Based Money Laundering enables criminal organizations to transfer illicit value through apparently legitimate trade transactions while avoiding direct movement of cash. Common TBML techniques include over-invoicing, under-invoicing, multiple invoicing,

false description of goods, phantom shipments, and the use of shell entities to manipulate commercial trade records. Since these activities are embedded within genuine trade operations involving multiple organizations and jurisdictions, detecting suspicious transactional behavior becomes highly complex for financial institutions and regulatory agencies.

Reports published by Global Financial Integrity indicate that trade misinvoicing and related TBML activities account for nearly 87% of illicit financial flows originating from developing economies [5]. Similarly, industry analyses estimate that approximately $1.6 trillion to $2 trillion in illicit value is concealed annually within trade invoices and commercial documentation [1]. These findings demonstrate the growing dependence of organized criminal networks on international trade systems for laundering illicit financial assets.

1.1.3 Problem Area

Conventional Anti-Money Laundering systems primarily rely on rule-based monitoring frameworks, predefined transaction thresholds, and manual auditing procedures for identifying suspicious financial activities. Although such approaches are effective in detecting isolated anomalies, they often fail to identify hidden relationships among entities involved in coordinated laundering operations. Modern TBML networks operate through interconnected ecosystems involving importers, exporters, shell companies, offshore entities, logistics providers, and financial intermediaries distributed across multiple jurisdictions, making detection significantly more challenging.

Traditional machine learning approaches further suffer from limitations associated with isolated transaction analysis and lack of relational understanding. Since financial transactions are processed independently, existing systems are unable to effectively capture structural interactions and behavioral dependencies hidden within large-scale trade networks. As a result, conventional AML systems frequently generate high false positive rates, increased compliance overhead, and delayed investigation processes.

Another major challenge involves institutional privacy and regulatory compliance constraints. Financial organizations are often unable to share sensitive transactional data across institutions due to strict legal and regulatory restrictions. Consequently, centralized fraud detection architectures become impractical in real-world banking environments. These limitations highlight the need for intelligent, scalable, and privacy-preserving analytical frameworks capable of effectively identifying suspicious transactional behavior

across distributed financial ecosystems.

1.2 Literature Review

The increasing complexity of global financial systems and the rapid growth of international trade have motivated extensive research in Anti-Money Laundering (AML) and Trade-Based Money Laundering (TBML) detection systems. Conventional AML frameworks primarily relied on rule-based transaction monitoring and manual auditing procedures to identify suspicious financial activities. However, the emergence of interconnected financial ecosystems, cross-border trade mechanisms, and sophisticated laundering strategies has significantly reduced the effectiveness of traditional approaches. Consequently, recent research has increasingly focused on integrating machine learning, graph analytics, and distributed intelligence techniques into modern AML systems.

Jiani Fan, Lwin Khin Shar, Ruichen Zhang, Ziyao Liu, Wenzhuo Yang, Dusit Niyato, and Kwok-Yan Lam [6] presented a comprehensive survey on deep learning-based Anti-Money Laundering frameworks for mobile financial systems. The study explored the application of Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Graph Neural Networks (GNNs) for transaction anomaly detection and proposed a Context-Risk-Predict framework for improving AML intelligence. The authors demonstrated that deep learning architectures significantly improve predictive performance compared to conventional rule-based systems. However, the framework primarily focused on centralized transaction processing environments and did not address institutional privacy constraints or distributed financial data challenges.

Morten Hjelholt Jensen and Alexandros Iosifidis [7] investigated the use of statistical analysis and supervised machine learning algorithms for financial anomaly detection. Their study evaluated models such as Logistic Regression, Random Forests, and Gradient Boosting techniques for suspicious transaction classification. Experimental results showed improved detection capability compared to static rule-based approaches. Nevertheless, the proposed methods analyzed transactions independently and lacked the ability to capture relational dependencies among interconnected financial entities involved in coordinated laundering operations.

Jinlong Song, Zhen Guo, Yutian Lei, and Minghao Zhao [8] proposed a dimension expansion framework for learning hidden money laundering activities within transaction networks. The framework enhanced transaction representations by integrating neighbor-

ing account information and contextual interaction features. The study demonstrated that relational feature aggregation improves the detection of concealed laundering patterns across financial networks. However, the model depended heavily on manually engineered relational features and lacked scalability for distributed cross-institutional financial environments.

In another study, Jinlong Song, Haoran Chen, Peng Xu, and Minghao Zhao [9] introduced a Social Attention Graph Neural Network framework for detecting illicit accounts in blockchain transaction systems. The proposed architecture transformed transaction ecosystems into graph structures and utilized attention-based graph learning mechanisms for anomaly classification. The study demonstrated that Graph Neural Networks outperform conventional machine learning models in identifying suspicious transactional behavior. However, the framework focused primarily on cryptocurrency transaction ecosystems and did not address invoice-level trade manipulations or heterogeneous commercial trade relationships associated with TBML activities.

Xiang Li, Zhen Wang, Yong Zhou, and Lei Zhang [10] proposed a Multi-View Graph-Based Hierarchical Representation Learning framework for organized money laundering group detection. Their work utilized multiple graph perspectives and hierarchical embedding mechanisms to identify coordinated laundering communities within financial transaction networks. Experimental findings demonstrated improved graph representation capability for identifying hidden laundering structures. However, the framework relied on predefined graph meta-path relationships and centralized graph repositories, limiting its applicability in privacy-sensitive financial environments.

Eren Gezici and Emre Sefer [11] proposed AMLPD, an unsupervised Anti-Money Laundering detection framework utilizing Personalized PageRank and graph diffusion techniques for deep vertex representation learning. The framework effectively captured hidden graph structures without requiring labeled datasets and demonstrated strong anomaly detection capability in sparse financial networks. However, the absence of supervised optimization mechanisms resulted in increased false positive classifications within highly imbalanced financial transaction datasets.

Mohammad Karim, Tariq Hassan, and Sameer Ali [12] investigated scalable semi-supervised graph learning architectures for large-scale financial transaction monitoring. Their study evaluated models such as SkipGCN and EvolveGCN for node classification

within AML transaction graphs. Experimental results demonstrated efficient scalability and improved anomaly detection performance in large graph repositories. Nevertheless, the framework assumed centralized access to financial transaction graphs and did not address institutional privacy and regulatory constraints associated with collaborative fraud detection systems.

Jongmin Koo, Hyun Park, and Seungwoo Kim [13] proposed an autoencoder-based suspicious transaction detection framework using Risk-Based Approaches (RBA) for banking systems. The proposed model reduced anomaly detection complexity by learning latent transactional representations from financial datasets. Although the framework improved internal banking surveillance efficiency, it focused primarily on point anomalies associated with individual accounts and failed to model interconnected laundering relationships distributed across complex trade ecosystems.

The literature survey indicates a clear transition from conventional rule-based AML systems toward machine learning, graph neural networks, and intelligent graph analytics for financial crime detection. Existing studies demonstrate that graph-based architectures significantly improve the ability to identify hidden transactional relationships and coordinated laundering structures compared to traditional machine learning techniques. However, several limitations remain insufficiently addressed, including heterogeneous graph modeling, real-time transaction analysis, institutional privacy preservation, and scalability across distributed financial environments. Most existing approaches either rely on centralized transaction repositories or focus primarily on banking or cryptocurrency transaction datasets without effectively modeling the complex trade relationships associated with Trade-Based Money Laundering activities.

Furthermore, many existing AML frameworks process financial transactions in isolation and fail to capture the interconnected dependencies among importers, exporters, shell entities, logistics providers, and trade documents distributed across multiple jurisdictions. The absence of privacy-preserving collaborative intelligence mechanisms further limits the practical deployment of such systems in real-world financial ecosystems where direct sharing of sensitive transactional data is restricted by regulatory policies. In addition, limited focus has been given to integrating heterogeneous graph learning, federated intelligence, and real-time anomaly detection within a unified Anti-Money Laundering framework. These research gaps highlight the need for an intelligent, scalable, and privacy-preserving

architecture capable of modeling complex trade ecosystems while supporting collaborative real-time TBML detection across distributed financial institutions.

1.3 Motivation

The work, *"Federated Heterogeneous Graph Neural Networks for Real-Time Detection of Trade-Based Money Laundering"*, was chosen to address the increasing complexity of financial crimes within modern global trade ecosystems through the development of an intelligent and privacy-preserving fraud detection framework. Conventional Anti-Money Laundering (AML) systems primarily rely on rule-based monitoring and isolated transaction analysis methods that are often ineffective in identifying hidden relationships and coordinated laundering activities distributed across interconnected financial networks. Although such systems are widely deployed in banking environments, they generate high false positive rates, require extensive manual investigation, and face significant challenges in detecting sophisticated Trade-Based Money Laundering (TBML) activities in real time.

The proposed work aims to overcome these limitations by developing a Federated Heterogeneous Graph Neural Network framework capable of modeling complex transactional relationships among financial entities while preserving institutional privacy. The integration of heterogeneous graph learning, graph attention mechanisms, and federated intelligence enables collaborative fraud analysis without direct sharing of sensitive transactional data across organizations. In addition, the framework supports automated Suspicious Transaction Report (STR) generation and dashboard-based visualization for real-time fraud monitoring and compliance analysis, demonstrating the feasibility of scalable and AI-driven Anti-Money Laundering systems for modern financial environments.

1.4 Problem Statement

Existing AML systems rely on rule-based monitoring and isolated transaction analysis, limiting their ability to detect complex Trade-Based Money Laundering (TBML) and hidden financial networks. Privacy and regulatory constraints further hinder collaborative fraud detection across institutions. Hence, a scalable and privacy-preserving framework is needed to model interconnected financial data and detect suspicious activities in real time.

1.5 Objectives

The objectives of the project are

1. To develop a Federated Heterogeneous Graph Neural Network framework for real-time detection of Trade-Based Money Laundering activities.
2. To model complex transactional relationships among financial entities using heterogeneous graph representation and graph attention mechanisms.
3. To generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions to support financial investigation and compliance processes.
4. To design an interactive dashboard for visualization of fraud predictions, transaction analysis, and real-time monitoring of suspicious activities.

1.6 Brief Methodology of the project

The proposed methodology focuses on developing a privacy-preserving and graph-based framework for real-time Trade-Based Money Laundering (TBML) detection. Initially, financial transaction data, trade records, customer information, and entity relationships are collected and preprocessed to remove inconsistencies and prepare structured datasets for analysis. The processed data is then transformed into a heterogeneous graph structure where entities such as accounts, organizations, invoices, and transactions are represented as interconnected nodes and edges.

Graph Attention-based learning mechanisms are employed to capture hidden relational dependencies and suspicious transactional patterns within the financial ecosystem. To preserve institutional privacy, federated learning is integrated into the framework, enabling collaborative model training across distributed financial environments without direct sharing of sensitive transactional data. The detected suspicious activities are further utilized for automated Suspicious Transaction Report (STR) generation and visualization through an interactive dashboard for real-time monitoring and fraud analysis.

1.7 Assumptions and Constraints of the Work

Assumptions

- The financial transaction records, trade invoices, and customer-related information used for analysis are assumed to be sufficiently structured, consistent, and prepro-

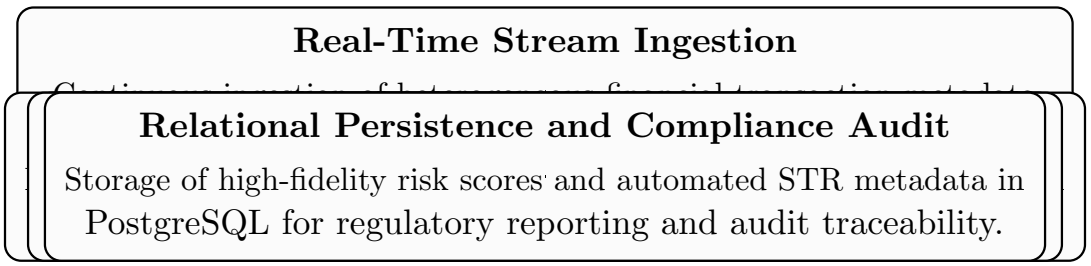

Figure 1.1: Proposed Methodology for TBML Detection

cessed for effective model training and evaluation.

- It is assumed that participating financial institutions maintain similar transactional formats and standardized trade-related attributes required for graph construction and suspicious activity analysis.

- The generated financial datasets are assumed to contain sufficient normal and suspicious transactional patterns necessary for learning hidden behavioral relationships associated with Trade-Based Money Laundering activities.

- It is assumed that heterogeneous graph representations can effectively model relationships among accounts, organizations, transactions, invoices, and trade entities for identifying suspicious financial behavior.

Constraints

- Limited access to real-world Trade-Based Money Laundering datasets due to institutional privacy regulations and confidentiality policies may affect the diversity of transactional data used for evaluation.

- Large-scale graph construction and real-time transaction analysis may increase computational complexity and memory requirements during fraud detection operations.

- The performance of the proposed framework depends significantly on the quality, completeness, and accuracy of the financial data available for model training and testing.

- The proposed framework is validated primarily under controlled academic and experimental conditions and may require additional optimization for large-scale industrial deployment.

1.8 Organization of the Report

This report is organized as follows.

- Chapter 2 discusses the theoretical concepts related to Anti-Money Laundering systems, Trade-Based Money Laundering, Graph Neural Networks, heterogeneous graph learning, and federated learning techniques used in the proposed work.
- Chapter 3 discusses the Software Requirement Specification (SRS) of the proposed system, including functional requirements, performance requirements, software and hardware requirements, and system constraints.
- Chapter 4 discusses the high-level architecture and overall system design of the proposed framework, including workflow diagrams, system components, dashboard architecture, and data flow representation.
- Chapter 5 discusses the detailed design of the proposed system, including graph construction methodology, fraud detection modules, STR generation workflow, and dashboard integration.
- Chapter 6 discusses the implementation details of the proposed framework, including dataset preprocessing, graph generation, backend integration, model development, and dashboard implementation.
- Chapter 7 discusses the testing and validation procedures performed on the proposed system, including functional testing, fraud prediction analysis, dashboard testing, and model evaluation techniques.
- Chapter 8 discusses the experimental results and performance evaluation of the proposed framework using various fraud detection metrics and comparative analysis.
- Chapter 9 discusses the conclusion of the proposed work, existing limitations, and possible future enhancements for improving the framework.

Chapter 2

Theory and Fundamentals of Project

CHAPTER 2

THEORY AND FUNDAMENTALS OF PROJECT

Developing an intelligent framework for Trade-Based Money Laundering detection requires a strong foundation in financial crime analysis, graph theory, and machine learning techniques. This chapter presents the fundamental concepts associated with Trade-Based Money Laundering, Graph Neural Networks, Graph Attention mechanisms, heterogeneous graph representation, and federated learning. The chapter further discusses the computational principles involved in relational transaction modeling, privacy-preserving learning, and real-time fraud analytics. These concepts collectively form the theoretical foundation for the proposed fraud detection framework.

2.1 Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) is a sophisticated form of financial crime in which illicit funds are transferred and concealed through seemingly legitimate international trade activities. Unlike conventional laundering methods that primarily depend on direct financial transactions, TBML exploits import-export systems, trade invoices, shipping documents, and cross-border commercial operations to disguise the movement of illegal funds. Due to the increasing implementation of strict Anti-Money Laundering (AML) regulations and Know Your Customer (KYC) policies, criminal organizations have increasingly shifted toward trade-oriented laundering mechanisms that are comparatively difficult to detect using traditional monitoring systems.

The complexity of modern global trade ecosystems, combined with the involvement of multiple financial entities and transactional relationships, has made TBML detection a major challenge for financial institutions and regulatory agencies. Consequently, intelligent analytical techniques capable of identifying hidden transactional dependencies and suspicious financial behavior have become increasingly important in modern Anti-Money Laundering systems.

2.1.1 Introduction to TBML

Trade-Based Money Laundering primarily involves the manipulation of international trade transactions to illegally transfer value between entities located across different countries. Criminal organizations often misuse trade documentation, invoice values, shipment quantities, and commercial agreements to move illicit funds while making the transac-

tions appear legitimate within global financial systems. Since these laundering activities are embedded within regular import-export operations, identifying suspicious behavior becomes highly challenging for financial institutions and regulatory agencies.

In many TBML operations, colluding buyers and sellers intentionally manipulate the declared value of traded goods to transfer illegal financial assets across borders. Techniques such as over invoicing, under invoicing, phantom shipments, and multiple invoicing are commonly used to conceal the movement of illicit proceeds. Such laundering mechanisms exploit the complexity and high transactional volume of international trade systems, making traditional rule-based Anti-Money Laundering approaches comparatively ineffective in detecting hidden financial relationships.

Figure 2.1: Trade-Based Money Laundering through trade misinvoicing and open-account transactions

Source: Adapted from "The Role of Misinvoicing in the Money Laundering Cycle," ResearchGate.

Figure **??** illustrates a typical Trade-Based Money Laundering workflow involving collusion between exporters and importers across different jurisdictions.

2.1.2 Common Techniques Used in TBML

Trade-Based Money Laundering activities are generally performed through the manipulation of trade documentation, shipment details, pricing structures, and financial transactions associated with international trade operations. Criminal organizations exploit the complexity of global trade ecosystems to transfer illicit value across borders while making the transactions appear legitimate. Several laundering techniques are commonly used within modern trade systems to disguise the movement of illegal funds and avoid detection from financial monitoring authorities.

Over Invoicing

Over invoicing is a laundering technique in which the value of exported or imported goods is intentionally declared higher than the actual market value. In this process, the buyer transfers an inflated payment amount to the seller, allowing excess funds to be moved across borders under the appearance of legitimate trade transactions. Criminal organizations use this technique to transfer illicit financial assets while avoiding suspicion from conventional banking systems.

Under Invoicing

Under invoicing involves declaring a lower value for traded goods when compared to their actual market price. This technique allows the buyer or seller to secretly transfer additional financial value outside formal banking channels. Under invoicing is commonly used to evade taxes, conceal illegal profits, and facilitate hidden movement of illicit assets across jurisdictions.

Multiple Invoicing

Multiple invoicing refers to the repeated use of the same trade invoice to justify several financial transactions for a single shipment of goods. By duplicating invoices across multiple banking channels or financial institutions, criminal entities can move large amounts of illicit funds while creating the appearance of legitimate trade payments. Detecting such duplicated transactional behavior is challenging in traditional rule-based monitoring systems.

Phantom Shipments

Phantom shipment laundering occurs when financial transactions are conducted for goods that are never actually shipped or delivered. Fraudulent trade documentation and

fabricated shipping records are used to create the illusion of legitimate trade activity while transferring illicit funds between colluding entities. Since no physical goods are exchanged, identifying phantom shipment activities requires strong transactional verification and relational analysis mechanisms.

Misrepresentation of Goods

Misrepresentation of goods involves intentionally providing false descriptions regarding the type, quantity, quality, or category of traded products within commercial documents and invoices. Criminal organizations use this technique to manipulate trade valuations and conceal suspicious financial activities under legitimate business transactions. Such inconsistencies are difficult to identify using isolated transactional analysis methods due to the large volume and diversity of international trade operations.

2.1.3 Challenges in Detecting TBML

Detecting TBML activities is highly challenging due to the complexity and interconnected nature of modern international trade ecosystems. Unlike conventional financial fraud, TBML transactions are often embedded within legitimate commercial operations, making suspicious activities difficult to distinguish from regular trade behavior. Criminal organizations exploit multiple jurisdictions, shell companies, intermediaries, and layered transactional structures to conceal the movement of illicit funds within normal trade systems.

Traditional AML systems primarily depend on rule-based monitoring approaches and isolated transactional analysis techniques. Although these systems are capable of identifying predefined suspicious patterns, they often fail to capture hidden relational dependencies among entities involved in large-scale trade networks. The increasing volume of cross-border financial transactions and evolving laundering strategies further reduce the effectiveness of conventional fraud detection mechanisms.

Another major challenge involves institutional privacy and regulatory compliance constraints associated with financial data sharing. Many organizations are unable to directly exchange sensitive transactional information due to confidentiality policies and legal restrictions. Consequently, modern TBML detection systems require intelligent, scalable, and privacy-preserving analytical frameworks capable of modeling complex financial relationships while supporting real-time fraud detection within large trade ecosystems.

2.2 Algorithms Used

2.2.1 Graph Attention Network v2 (GATv2)

Introduction

Graph Attention Network v2 (GATv2) is an advanced Graph Neural Network (GNN) architecture designed to improve the attention mechanism used in graph-based deep learning models. It was introduced as an enhancement over the original Graph Attention Network (GAT) to overcome the limitation of static attention and provide a more expressive and dynamic way of learning relationships between nodes in a graph.

GATv2 enables the model to automatically determine the importance of neighboring nodes while updating node representations. This makes it highly effective for applications such as:

- Traffic prediction
- Social network analysis
- Recommendation systems
- Molecular property prediction
- Fraud detection
- Autonomous vehicle navigation

Unlike traditional neural networks that work on Euclidean data such as images or sequences, GATv2 is specifically designed to process graph-structured data where relationships between entities are represented as nodes and edges.

Fundamental Concept of GATv2

The primary objective of GATv2 is to learn node embeddings by aggregating information from neighboring nodes using an attention mechanism.

For a node i, the model computes:

1. Which neighboring nodes are important
2. How much importance each neighbor should receive
3. A weighted combination of neighboring features

This process allows the network to dynamically focus on the most relevant neighbors.

Architecture and Working of GATv2

The overall architecture and workflow of GATv2 are shown in Figure 1.1.

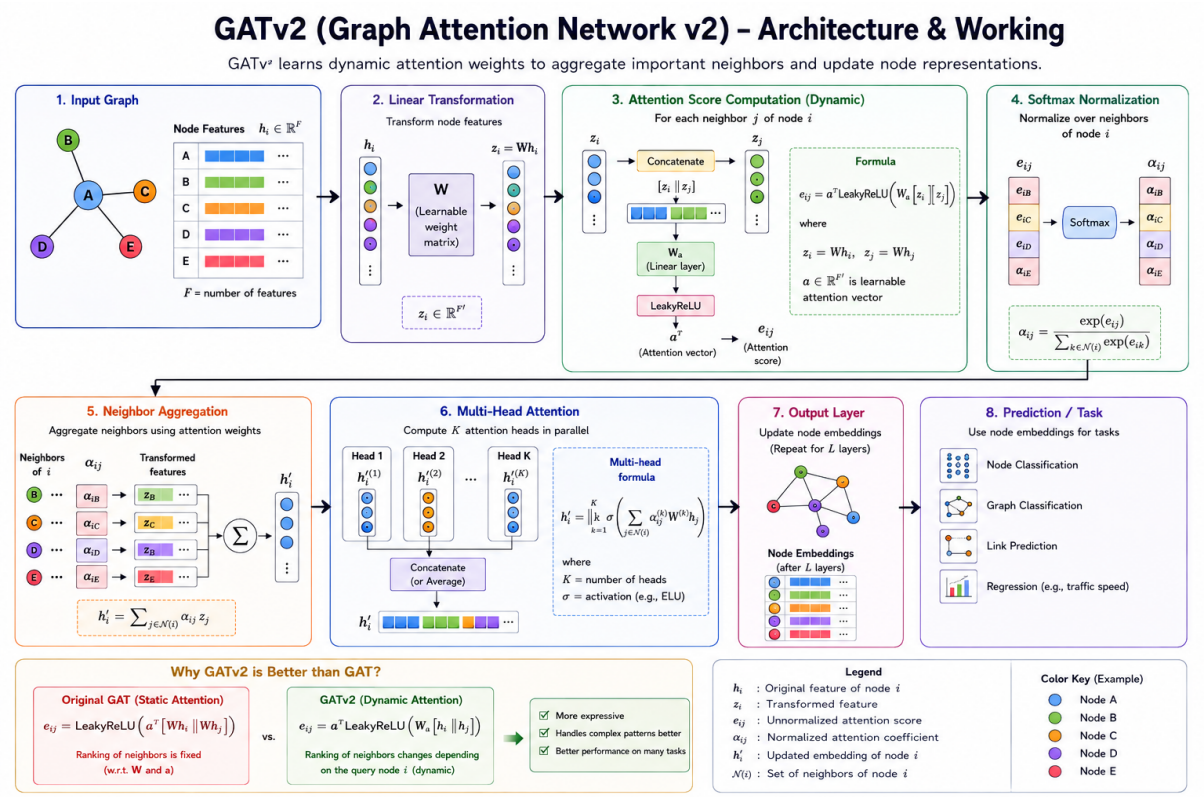

Figure 2.2: GATv2 Architecture and Working

The architecture of GATv2 consists of multiple stages including input graph processing, feature transformation, attention computation, normalization, neighbor aggregation, multi-head attention, and prediction generation. Each stage plays a significant role in learning effective graph representations.

Step 1: Input Graph The process begins with a graph consisting of nodes, edges, and node feature vectors. Each node contains a feature vector representing its properties.

Mathematically, the node feature vector is represented as:

$$h_i \in R^F$$

where:

- h_i represents the feature vector of node i
- F represents the number of input features

In traffic prediction systems, nodes may represent road intersections while edges represent road connectivity. Features may include traffic density, vehicle speed, congestion level, and signal timing information.

Step 2: Linear Transformation The input node features are transformed using a learnable weight matrix to extract higher-level representations.

$$z_i = W h_i$$

where:

- W is the learnable weight matrix
- h_i is the original node feature
- z_i is the transformed feature representation

This transformation helps the network learn useful combinations of input features and improves feature representation before attention computation.

Step 3: Attention Score Computation The attention mechanism is the core component of GATv2. For every connected pair of nodes (i, j), the model computes an attention score representing the importance of neighboring node j to node i.

The attention mechanism is given by:

$$e_{ij} = a^T \text{LeakyReLU}\left(W[h_i \| h_j]\right)$$

where:

- h_i and h_j are node feature vectors
- $[h_i \| h_j]$ represents concatenation
- W is the learnable weight matrix
- a is the learnable attention vector
- e_{ij} is the unnormalized attention score

This mechanism dynamically determines the importance of neighboring nodes.

Step 4: Softmax Normalization The attention scores are normalized using the softmax function to convert them into probability values.

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

where:

- α_{ij} is the normalized attention coefficient
- $\mathcal{N}(i)$ represents the neighboring nodes of node i

This normalization ensures that the attention coefficients sum to 1 and represent the relative importance of neighboring nodes.

Step 5: Neighbor Aggregation The neighboring node features are aggregated using weighted summation based on the attention coefficients.

$$h'_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j$$

where:

- h'_i is the updated node representation
- α_{ij} is the attention coefficient
- Wh_j is the transformed neighboring feature

This step allows important neighbors to contribute more significantly to the updated node embedding.

Step 6: Multi-Head Attention GATv2 uses multiple attention heads operating in parallel. Each attention head learns different relationships and feature interactions within the graph.

The multi-head attention mechanism is represented as:

$$h'_i = \Big\Vert_{k=1}^{K} \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} h_j \right)$$

where:

- K is the number of attention heads
- σ is the activation function
- $\|$ denotes concatenation

Multi-head attention improves learning capability by capturing multiple feature relationships simultaneously.

Step 7: Output Layer and Prediction The final node embeddings generated by the attention layers are passed to the output layer for downstream tasks such as:

- Node classification
- Graph classification
- Link prediction
- Regression tasks

The learned node embeddings contain rich structural and semantic information that helps improve prediction accuracy.

Advantages of GATv2

The major advantages of GATv2 are as follows:

- Dynamic attention mechanism enables adaptive neighbor importance learning.
- Improved expressiveness compared to the original GAT architecture.
- Automatically identifies important neighboring nodes without manual feature engineering.
- Multi-head attention improves robustness and representation learning.
- Performs effectively on irregular graph-structured data.
- Suitable for applications such as traffic prediction, recommendation systems, and autonomous navigation.

Limitations of GATv2

Despite its advantages, GATv2 also has certain limitations:

- Computational complexity increases for large-scale graphs.
- Requires higher memory consumption due to attention computations.
- Training time increases with the number of nodes and edges.
- Deep GATv2 architectures may suffer from over-smoothing problems.
- Scalability becomes challenging for extremely large graph datasets.

GATv2 vs GAT

Graph Attention Network v2 (GATv2) was introduced as an improvement over the original Graph Attention Network (GAT) to overcome the limitation of static attention mechanisms. Although both architectures use attention-based aggregation for graph representation learning, GATv2 provides a more expressive and dynamic attention mechanism.

In the original GAT architecture, the ranking of neighboring nodes becomes fixed after the linear transformation step. This limitation reduces the flexibility of the model in learning complex relationships between nodes. The attention mechanism in GAT is given by:

$$e_{ij} = \text{LeakyReLU}\left(a^T[Wh_i \| Wh_j]\right)$$

In GATv2, the order of operations is modified to make the attention mechanism dynamic and more expressive:

$$e_{ij} = a^T \text{LeakyReLU}\left(W[h_i \| h_j]\right)$$

This modification enables the attention scores to depend dynamically on both source and neighboring nodes, allowing the model to learn more meaningful node relationships.

The major differences between GAT and GATv2 are summarized in Table **??**.

Table 2.1: Comparison Between GAT and GATv2

Feature	**GAT**	**GATv2**
Attention Type	Static Attention	Dynamic Attention
Neighbor Ranking	Fixed after transformation	Changes dynamically based on node context
Expressiveness	Limited	More expressive
Feature Interaction	Moderate	Improved feature interaction learning
Learning Capability	Lower compared to GATv2	Better representation learning
Performance	Good	Improved performance on many graph tasks
Flexibility	Less flexible	More adaptive and flexible

GATv2 is preferred over GAT because it provides a more powerful and adaptive attention mechanism capable of learning complex graph relationships more effectively. The dynamic attention mechanism allows the model to assign different importance values to neighboring nodes depending on the context of the current node. This improves feature aggregation, representation learning, and prediction accuracy.

Due to these advantages, GATv2 is widely preferred in modern graph-based applications such as traffic prediction, autonomous vehicle navigation, recommendation systems, and social network analysis.

2.2.2 Long Short-Term Memory (LSTM)

Introduction to LSTM

Shortly after the first Elman-style Recurrent Neural Networks (RNNs) were trained using backpropagation, researchers observed a major limitation in learning long-term dependencies. This problem mainly occurred because of:

- Vanishing gradients
- Exploding gradients

Although gradient clipping could help reduce exploding gradients, handling vanishing gradients required a more advanced solution. To solve this issue, Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) architecture in 1997.

Unlike standard RNNs, where each recurrent node simply passes information to the next time step, LSTMs introduce a special structure called a memory cell. This memory

cell maintains an internal state that allows information to persist over many time steps without vanishing.

The key idea behind LSTM is the ability to:

- Remember important information
- Forget irrelevant information
- Control the flow of information through gates

This enables LSTMs to learn long-term sequential dependencies effectively.

Gated Memory Cell

Each memory cell is equipped with an internal state and a number of multiplicative gates that determine whether:

1. A given input should impact the internal state (the input gate),
2. The internal state should be flushed to 0 (the forget gate), and
3. The internal state of a given neuron should be allowed to impact the cell's output (the output gate).

Gated Hidden State

The key distinction between vanilla RNNs and LSTMs is that the latter support gating of the hidden state. This means that we have dedicated mechanisms for when a hidden state should be updated and also for when it should be reset. These mechanisms are learned and they address the concerns listed above. For instance, if the first token is of great importance we will learn not to update the hidden state after the first observation. Likewise, we will learn to skip irrelevant temporary observations. Last, we will learn to reset the latent state whenever needed. We discuss this in detail below.

Input Gate, Forget Gate, and Output Gate

The data feeding into the LSTM gates are the input at the current time step and the hidden state of the previous time step, as illustrated in Fig. 10.1.1. Three fully connected layers with sigmoid activation functions compute the values of the input, forget, and output gates. As a result of the sigmoid activation, all values of the three gates are in the range of $(0, 1)$.

Additionally, we require an input node, typically computed with a `tanh` activation function. Intuitively, the input gate determines how much of the input node's value should be added to the current memory cell internal state. The forget gate determines whether to keep the current value of the memory or flush it. The output gate determines whether the memory cell should influence the output at the current time step.

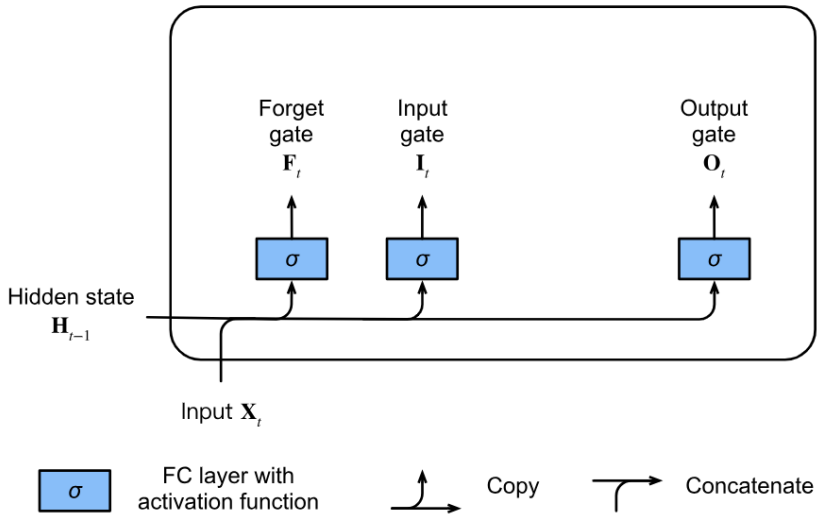

Figure 2.3: Computing the input gate, the forget gate, and the output gate in an LSTM model

Mathematically, suppose that there are h hidden units, the batch size is n, and the number of inputs is d. Thus, the input is $\mathbf{X}_t \in R^{n\times d}$ and the hidden state of the previous time step is $\mathbf{H}_{t-1} \in R^{n\times h}$.

Correspondingly, the gates at time step t are defined as follows: the input gate is $\mathbf{I}_t \in R^{n\times h}$, the forget gate is $\mathbf{F}_t \in R^{n\times h}$, and the output gate is $\mathbf{O}_t \in R^{n\times h}$. They are calculated as follows:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t\mathbf{W}_{\text{xi}} + \mathbf{H}_{t-1}\mathbf{W}_{\text{hi}} + \mathbf{b}_{\text{i}}),\\ \mathbf{F}_t &= \sigma(\mathbf{X}_t\mathbf{W}_{\text{xf}} + \mathbf{H}_{t-1}\mathbf{W}_{\text{hf}} + \mathbf{b}_{\text{f}}),\\ \mathbf{O}_t &= \sigma(\mathbf{X}_t\mathbf{W}_{\text{xo}} + \mathbf{H}_{t-1}\mathbf{W}_{\text{ho}} + \mathbf{b}_{\text{o}})\end{aligned} \tag{2.1}$$

where $\mathbf{W}_{\text{xi}}, \mathbf{W}_{\text{xf}}, \mathbf{W}_{\text{xo}} \in R^{d\times h}$ and $\mathbf{W}_{\text{hi}}, \mathbf{W}_{\text{hf}}, \mathbf{W}_{\text{ho}} \in R^{h\times h}$ are weight parameters and $\mathbf{b}_{\text{i}}, \mathbf{b}_{\text{f}}, \mathbf{b}_{\text{o}} \in R^{1\times h}$ are bias parameters.

Note that broadcasting is triggered during the summation. We use sigmoid functions to map the input values to the interval $(0, 1)$.

Input Node

Next we design the memory cell. Since we have not specified the action of the various gates yet, we first introduce the input node $\tilde{\mathbf{C}}_t \in R^{n \times h}$. Its computation is similar to that of the three gates described above, but uses a `tanh` function with a value range for $(-1, 1)$ as the activation function. This leads to the following equation at time step t:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{\text{xc}} + \mathbf{H}_{t-1} \mathbf{W}_{\text{hc}} + \mathbf{b}_{\text{c}}) \tag{2.2}$$

where $\mathbf{W}_{\text{xc}} \in R^{d \times h}$ and $\mathbf{W}_{\text{hc}} \in R^{h \times h}$ are weight parameters and $\mathbf{b}_{\text{c}} \in R^{1 \times h}$ is a bias parameter.

A quick illustration of the input node is shown in Fig. 10.1.2.

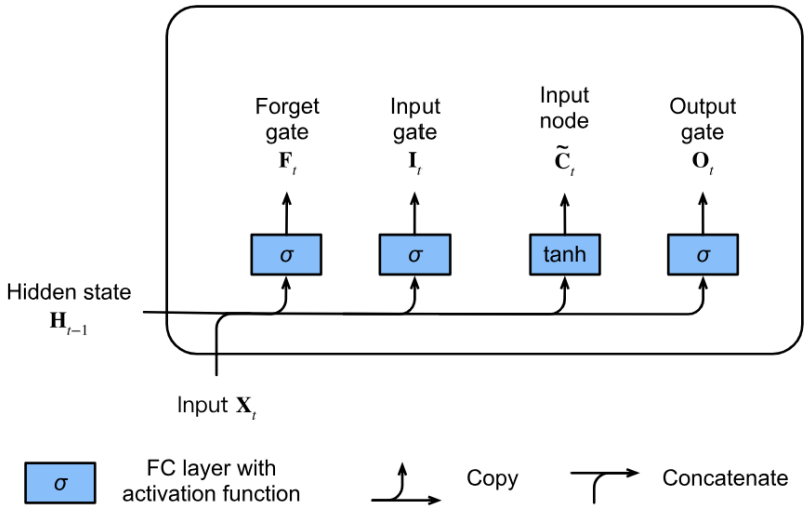

Figure 2.4: Computing the input node in an LSTM model

Memory Cell Internal State

In LSTMs, the input gate $\mathbf{I}_t$ governs how much we take new data into account via $\tilde{\mathbf{C}}_t$ and the forget gate $\mathbf{F}_t$ addresses how much of the old cell internal state $\mathbf{C}_{t-1} \in R^{n \times h}$ we retain. Using the Hadamard (elementwise) product operator $\odot$ we arrive at the following update equation:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \tag{2.3}$$

If the forget gate is always 1 and the input gate is always 0, the memory cell internal state $\mathbf{C}_{t-1}$ will remain constant forever, passing unchanged to each subsequent time step. However, input gates and forget gates give the model the flexibility of being able to learn

when to keep this value unchanged and when to perturb it in response to subsequent inputs.

In practice, this design alleviates the vanishing gradient problem, resulting in models that are much easier to train, especially when facing datasets with long sequence lengths.

We thus arrive at the flow diagram in Fig. 10.1.3.

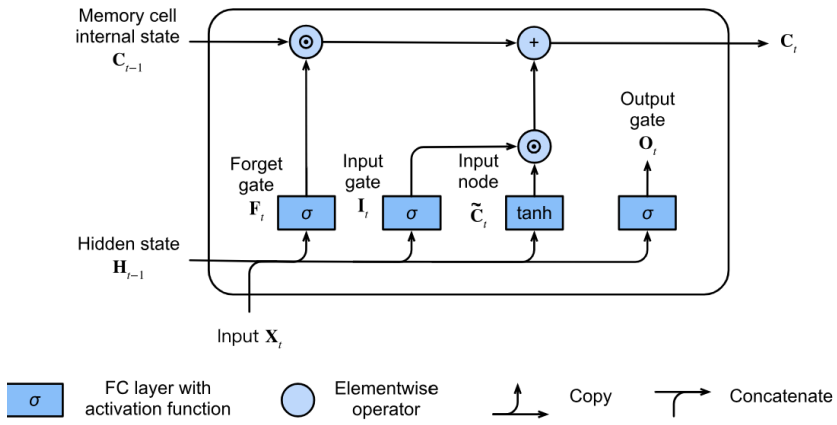

Figure 2.5: Computing the memory cell internal state in an LSTM model

Hidden State

Last, we need to define how to compute the output of the memory cell, i.e., the hidden state $\mathbf{H}_t \in R^{n \times h}$, as seen by other layers. This is where the output gate comes into play.

In LSTMs, we first apply `tanh` to the memory cell internal state and then apply another point-wise multiplication, this time with the output gate. This ensures that the values of $\mathbf{H}_t$ are always in the interval $(-1, 1)$:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \tag{2.4}$$

Whenever the output gate is close to 1, we allow the memory cell internal state to impact the subsequent layers uninhibited, whereas for output gate values close to 0, we prevent the current memory from impacting other layers of the network at the current time step.

Note that a memory cell can accrue information across many time steps without impacting the rest of the network (as long as the output gate takes values close to 0), and then suddenly impact the network at a subsequent time step as soon as the output gate flips from values close to 0 to values close to 1.

Fig. 10.1.4 has a graphical illustration of the data flow.

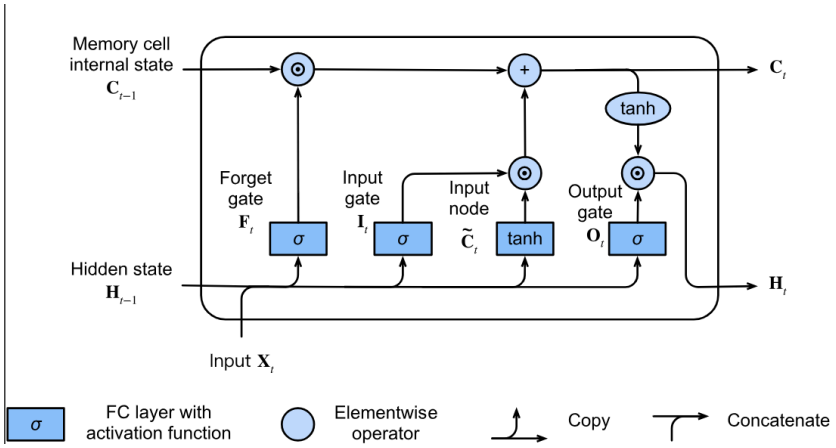

Figure 2.6: Computing the hidden state in an LSTM model

Advantages of LSTM

The major advantages of LSTM are as follows:

- LSTM effectively solves the vanishing gradient problem faced by traditional Recurrent Neural Networks (RNNs).
- It can learn long-term dependencies and preserve information over extended sequence lengths.
- The gating mechanism enables selective memory retention and controlled information flow.
- LSTM performs well on sequential and time-series data.
- It is highly effective in applications such as natural language processing, speech recognition, machine translation, traffic prediction, and stock market forecasting.
- The memory cell architecture allows important information to persist across multiple time steps.
- LSTM networks are more stable and easier to train compared to vanilla RNNs.

Limitations of LSTM

Despite its advantages, LSTM also has certain limitations:

- LSTM networks are computationally expensive due to the presence of multiple gates and memory cells.

- Training LSTM models requires high computational power and longer training time.
- The architecture is more complex compared to traditional RNNs.
- LSTMs require a large amount of training data for better generalization.
- They may suffer from overfitting when trained on small datasets.
- Parallel processing is difficult because sequential data must be processed step-by-step.
- For extremely long sequences, performance may still degrade despite improved memory retention.

Source: Adapted from "Dive into Deep Learning,Long Short-Term Memory (LSTM)."

2.3 Federated Learning

Federated Learning (FL) is a decentralized machine learning paradigm that enables multiple devices or organizations to collaboratively train a shared machine learning model without directly exchanging their raw data. Instead of transferring sensitive information to a centralized server, each participating client performs training locally on its own dataset and only shares model parameters such as weights and gradients with a central aggregation server. The server then combines these locally trained models to generate a global model that benefits from the collective knowledge of all participating clients.

The primary objective of FL is to preserve data privacy while still enabling collaborative intelligence across distributed systems. This approach is especially beneficial in domains where sensitive information cannot be centrally stored due to privacy, legal, or organizational constraints. Unlike traditional distributed learning, where local datasets are assumed to be independent and identically distributed (i.i.d.) and approximately equal in size, FL operates effectively even when the datasets are heterogeneous, non-i.i.d., and unevenly distributed among clients.

Another major distinction between federated learning and conventional distributed learning lies in the reliability and capability of participating nodes. In distributed learning, nodes are typically high-performance datacenters connected through high-speed communication networks. In contrast, FL environments commonly consist of mobile devices, IoT sensors, edge devices, or organizational servers that may have limited computational

power, unstable network connectivity, and energy constraints. Consequently, federated systems must handle issues such as client dropouts, communication delays, and varying hardware capabilities.

2.3.1 Introduction

Artificial Intelligence (AI) and Machine Learning (ML) technologies have experienced tremendous growth in recent years due to advancements in computational resources, availability of large-scale datasets, and improvements in deep learning algorithms. These technologies are now widely used in domains such as healthcare, banking, cybersecurity, transportation, e-commerce, smart homes, and financial monitoring systems. However, the rapid growth of machine learning applications has also introduced major concerns related to data privacy, security, and regulatory compliance.

Traditional machine learning systems generally rely on centralized training approaches where data collected from multiple users, institutions, or organizations is transferred to a centralized cloud server for model training. Although centralized learning often provides high model performance, it poses significant risks when handling sensitive and confidential information. In financial sectors, particularly in Trade-Based Money Laundering (TBML) detection systems, organizations such as banks, customs departments, shipping agencies, and financial institutions possess highly sensitive trade transaction records, invoice data, customer information, and cross-border payment details. Sharing such information with a centralized server may violate regulatory policies and expose organizations to security threats, financial fraud, and data breaches.

Federated Learning provides an effective solution to these challenges by enabling collaborative model training without requiring direct sharing of raw trade data. In a TBML detection environment, each participating institution trains a local machine learning model using its own transaction datasets, suspicious trade patterns, invoice records, and customer activity logs. Instead of sending raw data to a central server, only the locally trained model parameters are shared with the aggregation server. The central server combines the local model updates to create a global model capable of identifying complex money laundering activities across multiple organizations while maintaining data confidentiality.

FL has significantly improved privacy-preserving machine learning by ensuring that sensitive financial and trade-related information remains within the organization's local

infrastructure. This decentralized learning approach reduces the risks associated with centralized data storage and supports compliance with strict financial regulations and data protection laws such as GDPR, HIPAA, AML (Anti-Money Laundering) guidelines, and international financial monitoring standards.

In the context of TBML, federated learning enables multiple banks and financial institutions to collaboratively detect suspicious trade activities such as:

- Over-invoicing and under-invoicing of goods
- Multiple invoicing fraud
- Phantom shipments
- Trade diversion schemes
- Misrepresentation of product values
- Abnormal cross-border transaction patterns

Since money laundering activities often span across multiple countries and organizations, no single institution possesses sufficient data to identify all fraudulent patterns independently. FL allows these institutions to benefit from collective intelligence while ensuring that confidential customer and transaction data never leaves their local systems.

The overview of Federated Learning is shown in Fig. **??**. An FL system generally consists of three primary stakeholders:

1. The system owner or central learning coordinator
2. Contributor clients that locally train models using private datasets
3. User clients that utilize the final global model for prediction and analysis

The hardware infrastructure of FL systems mainly includes:

1. Central aggregation servers
2. Client devices or organizational servers

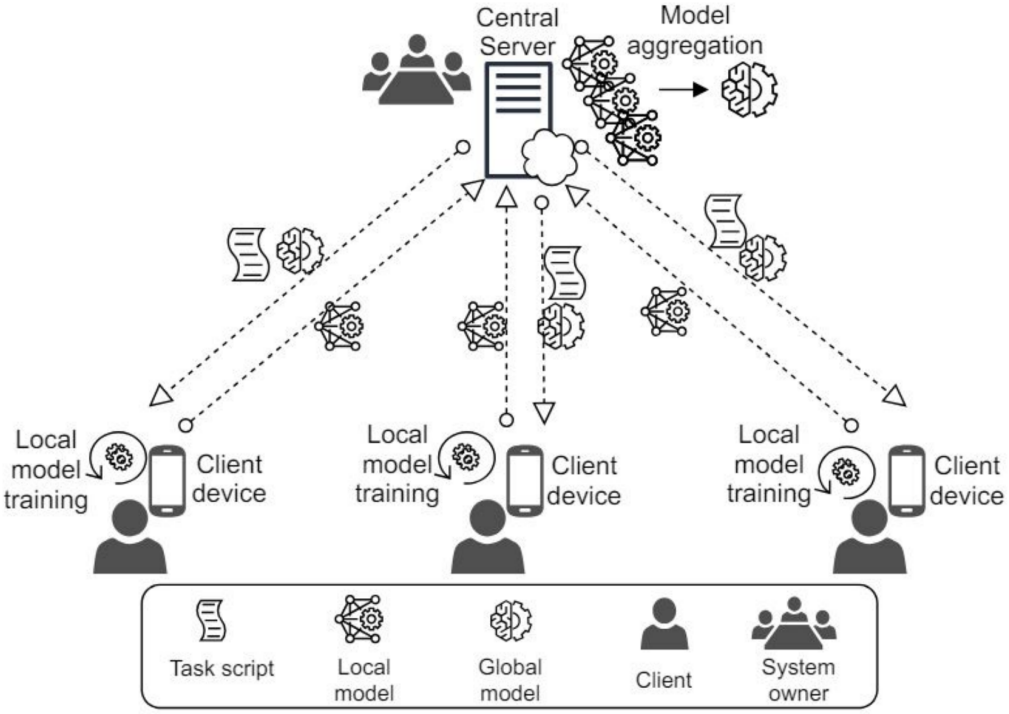

Figure 2.7: General Outlook of Federated Learning

Source: Adapted from "Principles and Components of Federated Learning Architectures" Arxiv."

Google originally introduced the concept of Federated Learning in 2016 through its Gboard mobile keyboard application, where multiple Android devices collaboratively improved typing prediction models without sharing private user text data. Since then, FL has emerged as a powerful paradigm for privacy-preserving collaborative learning across multiple industries.

In the field of Trade-Based Money Laundering detection, FL can revolutionize financial fraud analysis by enabling collaborative anomaly detection across banks, customs agencies, insurance companies, and regulatory authorities. Financial institutions can jointly train machine learning models to detect suspicious trading behavior, shell company activities, invoice manipulation, and hidden financial transactions while preserving customer confidentiality and organizational privacy.

Federated Learning also provides substantial advantages in terms of communication efficiency and bandwidth optimization. Instead of continuously transferring massive financial datasets to centralized servers, only compact model updates are communicated between clients and the central server. This significantly reduces network overhead and

operational costs while maintaining effective collaborative learning.

Furthermore, FL supports regulatory compliance requirements in highly sensitive financial domains. Many countries enforce strict regulations that prevent cross-border sharing of customer financial information. Federated Learning addresses this challenge by allowing organizations to participate in global model training without exposing confidential transactional data.

The fundamental operation of Federated Learning is based on two key processes:

1. Local model training on client devices or institutional servers
2. Global model aggregation at the central server

Each client trains a local model using its private dataset and periodically sends only the learned parameters to the aggregation server. The server combines the updates received from all participating clients using aggregation algorithms such as Federated Averaging (FedAvg) to generate an improved global model. This iterative training process continues until the model achieves satisfactory performance.

The general workflow of Federated Learning is illustrated in Fig. **??**.

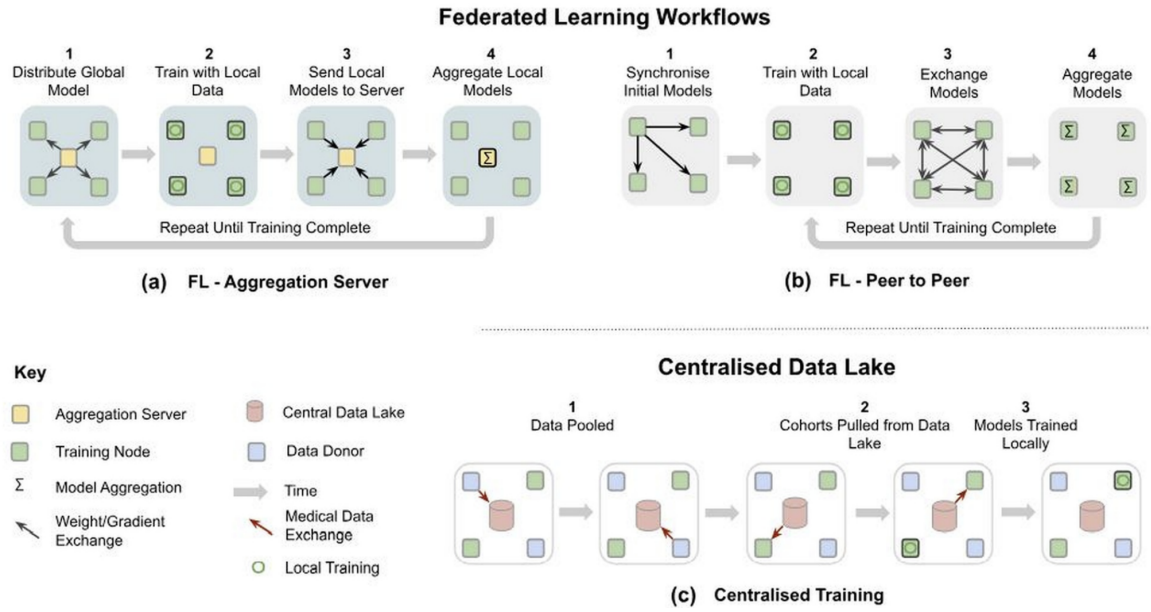

Figure 2.8: Federated Learning Workflows

Although Federated Learning provides major advantages in privacy preservation, communication efficiency, and collaborative intelligence, it also introduces several challenges. These include communication delays, heterogeneous client datasets, non-i.i.d. data distributions, client failures, model poisoning attacks, and secure aggregation complexities.

Despite these challenges, FL remains one of the most promising approaches for secure and privacy-aware machine learning in financial fraud detection systems such as Trade-Based Money Laundering analysis.

2.3.2 Working of Federated Learning

The iterative process of Federated Learning is composed of a sequence of client-server interactions known as *federated learning rounds*. In each round, the current global model is transmitted to selected client nodes, local training is performed independently on local datasets, and the generated local model updates are then aggregated to create an improved global model. This process continues iteratively until the desired model performance or convergence criteria are achieved.

In a standard Federated Learning environment, a central server coordinates the training process while local client devices or organizational servers perform local computations based on the instructions provided by the central server. However, alternative decentralized approaches also exist where clients communicate directly with one another using peer-to-peer communication, gossip protocols, or consensus-based methodologies without relying on a centralized coordinator.

Assuming one federated round corresponds to a single iteration of the collaborative learning process, the overall Federated Learning workflow can be summarized as follows.

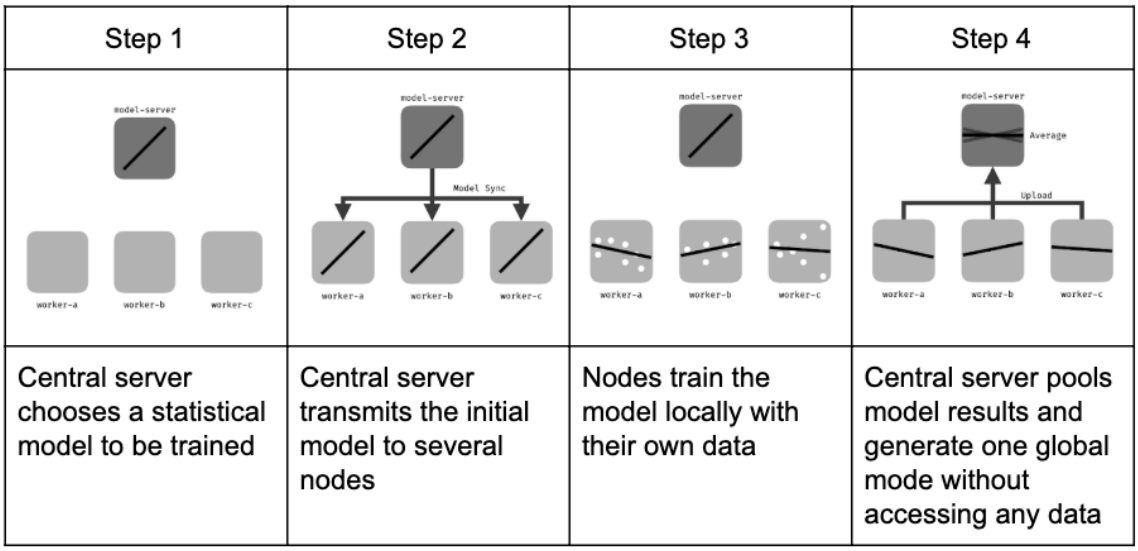

Figure 2.9: Federated Learning General Process in Central Orchestrator Setup

Source: Adapted from "Federated Learning" Wikipedia."

1. **Initialization:**

Based on the server requirements, an appropriate machine learning model such as linear regression, logistic regression, decision trees, neural networks, or boosting algorithms is selected and initialized. The client nodes are then activated and prepared to receive training instructions from the central server. In the context of Trade-Based Money Laundering (TBML), this initial model may be designed to detect suspicious trade transactions, abnormal invoice values, shell company activities, or hidden financial patterns.

2. **Client Selection:**

   During each federated learning round, only a subset of participating clients is selected for training. The selected nodes receive the latest global model parameters from the server while the remaining clients remain idle until future rounds. In TBML systems, selected clients may include banks, customs agencies, financial institutions, shipping companies, or trade monitoring authorities that possess relevant transactional datasets.

3. **Configuration:**

   The central server configures the local training process by specifying training parameters such as batch size, number of local epochs, learning rate, optimization method, and communication intervals. Each selected client then trains the received global model locally using its private datasets without sharing any raw transactional or customer information externally.

4. **Reporting and Aggregation:**

   After completing local training, each client transmits only the trained model parameters or gradients to the central aggregation server. The server aggregates the received local models using algorithms such as Federated Averaging (FedAvg) to generate an updated global model. The server also manages issues such as disconnected clients, delayed updates, or failed communications before initiating the next federated round.

   The Federated Averaging process can be mathematically represented as:

   :contentReference[oaicite:0]index=0

   where:

- w_{t+1} represents the updated global model,
- $w_k^{(t+1)}$ represents the locally trained model of client k,
- n_k represents the number of training samples at client k,
- n represents the total number of training samples across all clients.

5. **Termination:**

   The federated training process continues iteratively until a predefined stopping criterion is satisfied. Common termination conditions include reaching a maximum number of communication rounds, achieving a target accuracy level, minimizing loss values, or obtaining stable model convergence. Once the training process is completed, the final aggregated model is deployed for prediction and analysis tasks.

The workflow described above assumes synchronized model updates, where all participating clients complete local training before aggregation occurs. However, recent developments in Federated Learning have introduced asynchronous training approaches to address communication delays and varying computational capabilities among clients.

In asynchronous Federated Learning, clients can send model updates independently without waiting for all other clients to complete training. This approach significantly improves scalability and efficiency in large-scale distributed environments. Furthermore, advanced techniques such as *Split Learning* divide neural network computations across clients and servers, enabling intermediate neural network layers to be processed separately while reducing computational and communication overhead. Split learning can be applied during both training and inference phases in centralized as well as decentralized FL architectures.

In Trade-Based Money Laundering detection systems, asynchronous FL is highly beneficial because participating institutions may operate in different geographical regions, possess different computational resources, and experience varying communication delays. The flexibility of asynchronous training allows continuous collaborative learning without requiring strict synchronization among all participating organizations.

2.3.3 Advantages of Federated Learning

Federated Learning provides several significant advantages over traditional centralized machine learning approaches, particularly in domains involving highly sensitive in-

formation such as Trade-Based Money Laundering detection, healthcare, finance, and cybersecurity.

1. **Enhanced Data Privacy and Security:**
   Since raw data never leaves the local client devices or organizational servers, sensitive trade records, customer information, financial transactions, and invoice details remain protected within institutional boundaries. This significantly reduces the risks of data leakage and unauthorized access.

2. **Regulatory Compliance:**
   FL supports compliance with strict data protection regulations such as GDPR, HIPAA, AML guidelines, and financial privacy laws by eliminating the need for centralized storage of confidential data.

3. **Collaborative Intelligence Without Data Sharing:**
   Multiple organizations can collaboratively build highly accurate machine learning models without directly exchanging sensitive information. In TBML systems, banks and financial institutions can jointly detect hidden laundering patterns while preserving customer confidentiality.

4. **Reduced Communication of Raw Data:**
   Instead of transferring massive datasets across networks, only compact model parameters or gradients are exchanged between clients and the central server. This reduces network traffic and minimizes data transmission overhead.

5. **Scalability:**
   Federated Learning can efficiently support a large number of distributed devices and organizations across geographically diverse environments.

6. **Improved Model Generalization:**
   Since the global model learns from diverse datasets collected from multiple organizations and environments, it often achieves better generalization and robustness against unseen data patterns.

7. **Real-Time Learning at the Edge:**
   FL enables continuous local learning on edge devices such as smartphones, IoT

devices, and institutional servers without requiring permanent centralized connectivity.

2.3.4 Limitations of Federated Learning

Despite its advantages, Federated Learning also introduces several technical, statistical, and operational challenges.

Federated Learning requires frequent communication between client nodes and the central server during the learning process. Therefore, it demands sufficient local computational power, memory capacity, and high-bandwidth communication networks to efficiently exchange machine learning model parameters. Although FL avoids transmitting raw datasets, the repeated exchange of model updates can still impose significant communication overhead, especially in large-scale systems.

In practical environments, many participating devices such as smartphones, IoT devices, and edge systems are communication-constrained and often connected through unstable wireless networks such as Wi-Fi or mobile data connections. Consequently, standard FL mechanisms may not always be suitable without optimization techniques such as model compression, asynchronous communication, or adaptive client selection.

Federated Learning also raises several important statistical and operational challenges:

- **Data Heterogeneity:** Different clients may possess datasets with varying distributions, biases, and sizes, resulting in non-independent and non-identically distributed (non-i.i.d.) data.
- **Temporal Heterogeneity:** The statistical distribution of local datasets may change over time due to evolving user behavior, financial patterns, or operational conditions.
- **Interoperability Requirements:** Participating organizations must ensure compatibility between their datasets, data formats, and feature representations.
- **Data Maintenance and Curation:** Local datasets often require regular cleaning, labeling, and updating to maintain model quality.
- **Security Threats and Backdoor Attacks:** Since raw training data remains hidden, malicious clients may inject poisoned model updates or backdoor attacks into the global model.

- **Bias Detection Challenges:** The absence of centralized access to training data makes it difficult to identify and mitigate hidden biases related to age, gender, location, or demographic factors.
- **Node Failures and Communication Loss:** Partial or complete failure of participating clients can result in missing model updates and unstable global model convergence.
- **Lack of Labeled Data:** Some clients may not possess properly annotated or labeled datasets, affecting local model quality.
- **Heterogeneous Hardware Platforms:** Different client devices may have varying computational capabilities, operating systems, memory limitations, and processing speeds, complicating synchronized training.
- **Complex Coordination Mechanisms:** Efficient coordination between multiple distributed clients and aggregation servers requires sophisticated communication and synchronization protocols.

Although Federated Learning presents several challenges, continuous advancements in secure aggregation, communication optimization, differential privacy, and decentralized learning architectures are rapidly improving the practicality and effectiveness of FL systems for large-scale real-world applications such as Trade-Based Money Laundering detection.

2.4 User Interface

Modern financial monitoring platforms increasingly rely on scalable web application technologies to support real-time transaction analysis, fraud monitoring, and interactive visualization of financial activities. The rapid growth of digital banking and online financial services has significantly increased the demand for responsive, secure, and data-driven monitoring systems capable of handling large volumes of transactional information efficiently. Consequently, modern Anti-Money Laundering systems employ component-based web architectures and client-server communication models to provide efficient user interaction and real-time analytical capabilities.

Advanced frontend frameworks and backend service architectures play a critical role in enabling modular application development, dynamic rendering, secure authentication,

and seamless integration with fraud detection engines. Such technologies support the visualization of suspicious transactional activities, alert generation, role-based access control, and automated compliance workflows within financial institutions. The integration of scalable user interface technologies with intelligent analytical systems further enhances operational efficiency and supports real-time financial crime investigation processes.

2.4.1 Modern Web Application Frameworks

Modern web application frameworks provide the architectural foundation required for developing scalable, responsive, and interactive software systems. Traditional web applications primarily relied on static page rendering and tightly coupled server-side architectures, which limited dynamic user interaction and real-time data processing capabilities. In contrast, modern frameworks adopt component-based and asynchronous rendering approaches that improve application scalability, maintainability, and user experience within large-scale distributed systems.

Frontend frameworks such as React.js enable the development of reusable and modular user interface components capable of dynamically updating application states without requiring complete page reloads. Such frameworks employ virtual Document Object Model (DOM) mechanisms and state-driven rendering principles to efficiently manage user interactions and real-time data visualization. This architectural approach significantly improves responsiveness and enables scalable dashboard development for data-intensive applications such as financial monitoring systems.

Server-side rendering and hybrid rendering frameworks such as Next.js further enhance modern web application performance by combining client-side interactivity with optimized server-side content delivery. These frameworks support efficient routing, dynamic data fetching, and improved application scalability while reducing rendering overhead for large-scale systems. Consequently, modern web application frameworks have become an essential foundation for building secure, scalable, and real-time financial monitoring platforms.

2.4.2 Component Lifecycle and Reactivity

Modern web application frameworks employ reactive rendering mechanisms to dynamically synchronize the user interface with application data and user interactions. In component-based architectures such as React and Next.js, the user interface continuously

responds to changes in application state, backend responses, and client-side events. This reactive behavior enables efficient rendering of dynamic content while improving scalability, responsiveness, and maintainability within large-scale financial monitoring systems.

The component lifecycle in modern React and Next.js applications generally follows a sequence of rendering, interaction, updating, and cleanup stages. These stages collectively manage how components are initialized, rendered, updated, and removed during application execution.

Server-Side Rendering: Initially, application components are rendered within the server environment where transaction data, analytical summaries, and monitoring information are processed before transmitting the generated content to the client browser. This server-side rendering approach improves initial loading performance and reduces unnecessary client-side computational overhead.

Hydration and Client Initialization: After the rendered content reaches the client browser, React hydrates the interface by attaching event listeners and enabling interactive functionalities within client-side dashboard components. During this stage, local states, asynchronous API calls, and dynamic monitoring services are initialized to support user interaction and real-time transaction visualization.

Reactive State Updates: As users interact with the dashboard, component states and analytical data continuously update according to backend responses and fraud monitoring events. React utilizes Virtual DOM reconciliation mechanisms to selectively update only the affected dashboard components instead of re-rendering the complete interface.

Re-rendering and Synchronization: The framework dynamically synchronizes updated transaction information, fraud alerts, risk scores, and compliance monitoring analytics with the user interface whenever state transitions occur. This reactive rendering process enables efficient real-time visualization within the proposed Anti-Money Laundering monitoring framework.

Unmounting and Resource Cleanup: When components are removed from the active interface, allocated resources, temporary states, and subscriptions are automatically released to maintain application stability and efficient memory utilization.

Such lifecycle management principles are essential in modern financial monitoring

platforms where continuous transactional updates, alert generation, and real-time analytical visualization must be processed efficiently while ensuring scalable and responsive user interaction.

2.4.3 Data Visualization

Understanding the theoretical aspects of data visualization is important for analyzing large volumes of financial transactions and interpreting suspicious transactional behavior within real-time monitoring systems. Visualization of analytical outputs enables financial institutions and compliance analysts to identify fraud patterns, transaction anomalies, risk distributions, and suspicious entity relationships more efficiently when compared with conventional tabular analysis methods. Interactive monitoring dashboards further improve situational awareness by enabling continuous observation of fraud alerts, transaction flows, and compliance activities without interrupting real-time analytical processes.

This work conceptualizes financial monitoring and fraud visualization through a web-based dashboard architecture developed using modern frontend technologies and analytical visualization frameworks.

Frontend Framework: Next.js is employed as the frontend framework for developing responsive and modular user interface components capable of supporting scalable financial monitoring applications. The framework supports server-side rendering, dynamic routing, and efficient integration with backend analytical services operating within the proposed TBML detection platform.

Analytical Visualization Components: Recharts is utilized for visualizing transactional analytics, fraud classifications, risk score distributions, and temporal financial trends through interactive line graphs, bar charts, pie charts, and statistical dashboards. These visualization mechanisms improve analytical interpretation and enable efficient monitoring of suspicious financial activities.

Role-Based Dashboard Interfaces: The dashboard architecture incorporates role-based access mechanisms to provide controlled access to fraud investigation workflows, Suspicious Transaction Report generation, real-time monitoring services, and institutional compliance functionalities according to user privileges and operational responsibilities.

Interactive Monitoring Services: Interactive analytical panels support continuous

monitoring of suspicious transactions, transaction histories, fraud alerts, and prediction results generated by intelligent detection modules operating within the framework. The visualization environment additionally supports filtering, notification handling, and compliance-oriented monitoring operations for financial investigators and regulatory analysts.

Modern visualization systems additionally support analytical filtering, export functionalities, and alert management services that assist investigators and compliance officers in conducting financial analysis and generating compliance reports for further examination. The detailed implementation and integration of these visualization technologies within the proposed TBML detection framework are discussed further in the system implementation chapter.

2.5 Summary

In this chapter, the theoretical foundations required for understanding the proposed Trade-Based Money Laundering detection framework were presented. The chapter began with an introduction to Trade-Based Money Laundering mechanisms and common laundering techniques, establishing the financial and transactional context in which illicit fund movement occurs within international trade ecosystems.

The chapter further explored the challenges associated with detecting suspicious trade activities using conventional Anti-Money Laundering approaches. Concepts related to graph-based financial modeling, heterogeneous graph representation, Graph Neural Networks, Graph Attention mechanisms, and federated learning were discussed to provide the analytical and computational foundation required for intelligent and privacy-preserving fraud detection systems.

Finally, the chapter introduced the concepts associated with modern financial monitoring interfaces, reactive web architectures, and real-time data visualization systems used for fraud analytics and compliance monitoring. The theoretical concepts discussed throughout this chapter provide the foundation for understanding the system architecture, implementation methodologies, and analytical workflows presented in the subsequent chapters.

Chapter 3

Software Requirement Specification

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION

The development of an intelligent Trade-Based Money Laundering detection framework requires careful identification of the operational, computational, and functional requirements associated with the system. Since the proposed framework integrates graph-based fraud detection, federated learning, real-time monitoring, and compliance management mechanisms, defining the system requirements becomes essential for ensuring scalability, security, and efficient analytical performance. This chapter presents the Software Requirement Specification (SRS) of the proposed TBML detection framework, including the overall system description, product functionalities, user characteristics, functional and non-functional requirements, and the software and hardware resources required for implementing the proposed system.

3.1 Overall Description

The proposed system is designed as a privacy-preserving financial monitoring platform for detecting suspicious Trade-Based Money Laundering activities within large-scale transactional ecosystems. The framework combines heterogeneous graph-based fraud analysis, federated learning mechanisms, role-based monitoring dashboards, and automated Suspicious Transaction Report (STR) generation within a unified analytical environment. The system supports real-time transaction monitoring, fraud classification, and interactive visualization of suspicious financial activities to assist compliance analysts and financial institutions in identifying hidden laundering patterns efficiently.

3.1.1 Product Perspective

The proposed TBML detection framework functions as an intelligent financial monitoring and fraud analysis platform capable of integrating machine learning-based anomaly detection with interactive compliance monitoring systems. The architecture combines heterogeneous graph representation, Graph Neural Network-based analytical models, federated learning mechanisms, and modern web-based visualization technologies within a unified system environment.

The system represents financial entities such as accounts, organizations, invoices, and transactions as interconnected graph structures for relational analysis. Fraud detection modules analyze these relationships to identify suspicious transactional dependencies

and hidden laundering patterns that are difficult to detect using conventional rule-based Anti-Money Laundering systems. The framework additionally integrates role-based dashboards for transaction monitoring, fraud visualization, alert management, and automated compliance reporting within financial institutions.

3.1.2 Product Functions

The primary functionalities supported by the proposed framework are summarized as follows:

- **Transaction Monitoring:** The system continuously monitors financial transactions and trade-related activities to identify suspicious transactional behavior within large-scale financial ecosystems.
- **Graph-Based Financial Analysis:** Financial entities such as accounts, organizations, invoices, and transactions are represented as interconnected graph structures for relational analysis and hidden dependency detection.
- **Fraud Classification and Risk Prediction:** Graph Neural Network-based analytical models classify suspicious transactions and generate fraud risk predictions based on transactional relationships and behavioral patterns.
- **Federated Learning Support:** The framework supports collaborative model training across multiple institutional environments while preserving sensitive financial data privacy.
- **Real-Time Dashboard Visualization:** Interactive monitoring dashboards provide visualization of transaction analytics, fraud alerts, risk distributions, and suspicious activity summaries for compliance analysts.
- **Suspicious Transaction Report (STR) Generation:** The system automatically generates structured Suspicious Transaction Reports for identified fraudulent transactions to support compliance and regulatory workflows.
- **Authentication and Role-Based Access Control:** Secure authentication and controlled access mechanisms are incorporated to manage user privileges and protect sensitive financial information within the monitoring platform.

3.1.3 User Characteristics

The proposed TBML detection framework is designed to support role-based interaction for users involved in transaction monitoring, fraud investigation, and regulatory compliance activities. Since the system handles sensitive financial information and suspicious transactional records, controlled access mechanisms are incorporated to ensure secure and authorized usage of the platform.

The major user categories supported by the system are summarized as follows:

- **Bank Compliance Team:** The bank compliance team is responsible for monitoring institution-level transactions and analyzing suspicious transactional activities identified by the fraud detection system. Authorized users can review transaction details, examine analytical dashboards, verify fraud predictions, and classify suspicious activities for further investigation. In addition, the compliance team can escalate suspicious transactions as fraudulent, mark transactions for clearance, and generate Suspicious Transaction Reports (STRs) for confirmed fraud cases.

- **Regulatory Administrator:** The regulatory administrator functions as a higher-level monitoring authority with access to suspicious transaction reports generated by participating financial institutions. The administrator can review institution-level fraud cases, analyze consolidated STR records, monitor compliance activities across multiple banks, and oversee large-scale suspicious financial activities within the monitoring ecosystem.

3.1.4 Constraints

The development and deployment of the proposed TBML detection framework are subject to multiple operational, computational, and regulatory constraints associated with financial monitoring systems. These limitations influence the scalability, analytical performance, and real-time operational capabilities of the framework.

The major constraints associated with the proposed system are summarized as follows:

- **Data Privacy and Regulatory Constraints:** Financial institutions operate under strict privacy regulations and compliance policies that restrict direct sharing of sensitive transactional information across organizations.

- **Large-Scale Transactional Data Processing:** The system must process large volumes of interconnected financial transactions and graph relationships, which may increase computational complexity and memory utilization during real-time analysis.
- **Availability of Real-World TBML Datasets:** Access to publicly available Trade-Based Money Laundering datasets is limited due to confidentiality and regulatory restrictions, making large-scale real-world experimentation challenging.
- **Dynamic Fraud Patterns:** Money laundering strategies continuously evolve over time, requiring fraud detection models to adapt to changing transactional behaviors and emerging laundering techniques.
- **Infrastructure and Computational Limitations:** Training graph-based machine learning models and processing heterogeneous financial networks require significant computational resources, especially for large-scale transactional ecosystems.

3.2 Specific Requirements

The specific requirements of the proposed TBML detection framework define the operational functionalities, performance expectations, software dependencies, and computational resources required for efficient execution of the system. These requirements ensure that the framework can support real-time fraud analysis, scalable transaction monitoring, secure user interaction, and intelligent compliance management within financial institutions.

3.2.1 Functional Requirements

The functional requirements define the major operations and services that must be supported by the proposed TBML detection framework. The primary functional requirements of the system are summarized as follows:

- The system shall authenticate authorized users before providing access to monitoring dashboards and analytical services.
- The system shall support role-based access control for bank compliance teams and regulatory administrators.

- The system shall process transactional datasets and construct heterogeneous graph structures representing financial entities and transactional relationships.
- The system shall classify suspicious financial transactions using graph-based analytical models and generate fraud risk predictions.
- The system shall support real-time visualization of transaction analytics, fraud alerts, and suspicious activity summaries through interactive dashboards.
- The system shall allow bank compliance teams to review suspicious transactions and classify them for fraud escalation or clearance.
- The system shall automatically generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions.
- The system shall enable regulatory administrators to monitor fraud reports and suspicious activities generated across participating financial institutions.
- The system shall support analytical filtering and transaction search functionalities for efficient fraud investigation.
- The system shall maintain secure storage and controlled access to sensitive transactional and compliance-related information.

3.2.2 Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, and performance expectations associated with the proposed TBML detection framework. These requirements ensure that the system remains scalable, secure, reliable, and responsive while processing large-scale financial transactions and analytical workloads.

The primary non-functional requirements associated with the proposed system are summarized as follows:

- **Scalability:** The framework should support scalable processing of large transactional datasets and heterogeneous graph structures without significant degradation in analytical performance.

- **Real-Time Processing:** The system should support low-latency transaction analysis and fraud prediction to enable continuous real-time monitoring of suspicious financial activities.
- **Security and Privacy:** The framework should ensure secure access control, protected communication, and privacy-preserving handling of sensitive financial information and compliance-related data.
- **Reliability:** The system should maintain stable analytical performance and continuous monitoring capabilities during prolonged operational workloads and concurrent user access.
- **Availability:** The monitoring platform should provide consistent access to dashboards, fraud analytics, and reporting services for authorized institutional users.
- **Maintainability:** The system architecture should support modular development, efficient debugging, and simplified integration of additional analytical modules and monitoring functionalities.
- **Usability:** The dashboard interfaces and visualization systems should provide intuitive interaction mechanisms for compliance analysts and regulatory administrators during fraud investigation activities.

3.2.3 Software Requirements

The implementation of the proposed TBML detection framework requires multiple software technologies, machine learning libraries, backend services, database systems, and visualization frameworks to support graph-based fraud detection, federated learning, real-time transaction monitoring, and compliance management functionalities. The proposed framework integrates both analytical and monitoring components that require coordinated interaction between frontend dashboards, backend processing services, database systems, and machine learning environments. These technologies collectively support scalable fraud analysis, secure transaction management, and efficient visualization of suspicious financial activities. The software requirements associated with the proposed system are summarized in Table **??**.

Table 3.1: Software Requirements of the Proposed TBML Detection Framework

Category	**Technologies / Tools**	**Purpose in the System**
Operating System	Windows 11	Provides the development and deployment environment for backend services, databases, and analytical modules.
Programming Languages	Python, JavaScript	Used for backend development, machine learning workflows, API services, and frontend application development.
Frontend Frameworks	Next.js, React.js, Tailwind CSS	Supports responsive user interface development, dashboard visualization, and real-time monitoring functionalities.
Backend Framework	FastAPI	Facilitates asynchronous API communication between frontend dashboards and backend fraud detection services.
Database Systems	PostgreSQL, Memgraph	Stores transactional records, graph relationships, compliance information, and analytical data required for fraud analysis.
Machine Learning Frameworks	PyTorch, PyTorch Geometric	Supports deep learning operations, Graph Neural Network implementation, and graph-driven fraud analysis.
Federated Learning Framework	Flower	Enables distributed federated learning and privacy-preserving model aggregation across institutional environments.
Streaming Platform	Apache Kafka	Supports real-time transaction streaming and event-driven processing within the monitoring pipeline.
Containerization Platform	Docker	Provides containerized deployment and scalable execution environments for system integration.
LLM Integration Service	Groq LLM API	Supports automated generation of Suspicious Transaction Reports and compliance-oriented analytical summaries.

3.2.4 Hardware Requirements

The implementation of the proposed TBML detection framework requires the following hardware components to support graph processing, machine learning model execution, federated learning coordination, real-time transaction monitoring, and analytical dashboard visualization. The hardware requirements associated with the proposed system are summarized in Table **??**.

Table 3.2: Hardware Requirements of the Proposed TBML Detection Framework

Hardware Component	**Minimum Requirement**	**Purpose in the System**
Processor	Intel Core i7 / AMD Ryzen 7	Supports efficient execution of graph analytical models, backend services, and machine learning computations.
RAM	16 GB	Supports graph datasets, real-time analytical workloads, and concurrent monitoring operations.
GPU	NVIDIA RTX 3050	Accelerates Graph Neural Network training and deep learning computations.
Storage	512 GB SSD	Stores transactional datasets, graph databases, analytical models, and generated compliance reports.

3.3 Summary

In this chapter, the Software Requirement Specification of the proposed Trade-Based Money Laundering detection framework was presented. The operational functionalities, user roles, functional and non-functional requirements associated with the proposed fraud detection platform were discussed to establish the monitoring and analytical capabilities of the system.

The chapter also identified the software technologies, database systems, machine learning frameworks, and hardware resources required for implementing the proposed framework. These specifications collectively establish the operational and computational foundation required for developing the proposed TBML detection system, which is further discussed in the subsequent system design and implementation chapter.

Chapter 4

High Level Design

CHAPTER 4

HIGH LEVEL DESIGN

The design of the proposed Trade-Based Money Laundering detection framework focuses on developing a scalable, privacy-preserving, and intelligent financial monitoring system capable of analyzing complex transactional relationships within large-scale trade ecosystems. Since modern TBML activities involve interconnected financial entities, cross-border transactions, and evolving laundering strategies, the proposed framework integrates graph-based analytical models, federated learning mechanisms, and real-time monitoring services to support efficient fraud detection and compliance management. This chapter presents the major design considerations, architectural strategies, overall system architecture, and data flow mechanisms associated with the proposed fraud detection framework.

4.1 Design Considerations

The design of the proposed TBML detection framework is influenced by multiple operational, analytical, and security-related factors associated with large-scale financial monitoring systems. Since Trade-Based Money Laundering activities involve interconnected financial entities, cross-border transactions, and hidden transactional dependencies, the system architecture must support scalable analytical processing, secure institutional collaboration, and efficient graph-based fraud analysis. The major design considerations associated with the proposed framework are discussed in the following subsections.

4.1.1 General Considerations

The proposed framework is designed to support intelligent monitoring and analysis of large-scale financial transactions within distributed institutional environments. Since suspicious laundering activities often involve interconnected accounts, organizations, invoices, and transactional relationships, the system utilizes graph-based representation mechanisms to model hidden financial dependencies more effectively than conventional relational approaches. The framework additionally supports real-time transaction processing and scalable analytical operations to manage continuously growing financial datasets.

Security and privacy preservation also represent important considerations within the proposed system architecture. Financial institutions operate under strict regulatory policies that restrict unauthorized access and direct sharing of sensitive transactional infor-

mation. Consequently, the framework incorporates secure authentication mechanisms, role-based access control, and federated learning strategies to support protected institutional collaboration and privacy-preserving fraud analysis.

The system is additionally designed using a modular architecture to simplify integration, maintenance, and future scalability of analytical services. Independent modules associated with transaction monitoring, graph construction, fraud detection, compliance reporting, and dashboard visualization operate collaboratively within the overall monitoring framework while maintaining architectural flexibility for future enhancements.

4.1.2 Development Methods

The proposed TBML detection framework is developed using a modular and iterative development methodology to support scalable system integration and continuous refinement of analytical functionalities. Individual system components including transaction monitoring services, graph construction pipelines, fraud detection models, federated learning environments, and dashboard interfaces are developed and tested independently before integration within the complete monitoring framework.

The development process additionally incorporates incremental model evaluation and experimental refinement to improve fraud detection performance and analytical reliability. Graph-based machine learning models are trained using processed transactional datasets, while frontend and backend services are integrated through API-based communication mechanisms to support real-time interaction between monitoring modules and analytical services.

Version control and collaborative development practices are utilized throughout the implementation process to manage backend services, machine learning workflows, frontend interfaces, and analytical configurations efficiently. This modular development methodology simplifies debugging, testing, maintenance, and future enhancement of the proposed fraud detection framework.

4.2 Architectural Strategies

The architectural strategies adopted in the proposed TBML detection framework focus on supporting scalable financial transaction analysis, secure institutional collaboration, and efficient interaction between analytical services and monitoring interfaces. The framework integrates graph-based fraud detection models, backend analytical services,

real-time dashboards, and distributed learning mechanisms within a unified monitoring environment. The major architectural strategies associated with the proposed framework are discussed in the following subsections.

4.2.1 Programming Language

The proposed TBML detection framework primarily utilizes Python for backend services, graph processing, machine learning workflows, and analytical computations. Python was selected due to its extensive support for deep learning libraries, graph analytical frameworks, and federated learning environments. Libraries such as PyTorch, PyTorch Geometric, and Flower are utilized for implementing graph neural network models and distributed learning mechanisms within the fraud detection framework.

The frontend monitoring dashboard is developed using Next.js and React.js to support responsive user interface development and real-time analytical visualization. JavaScript and TypeScript are utilized for implementing dashboard components, transaction monitoring interfaces, and compliance management services. FastAPI is additionally integrated to support API-based communication between frontend dashboards, backend services, and machine learning modules.

4.2.2 System Integration Architecture

The proposed framework follows a modular client-server and service-oriented integration architecture to support coordinated interaction between analytical modules, backend services, and frontend monitoring interfaces. The fraud detection models, transaction monitoring services, authentication modules, and compliance reporting components operate as independent services while maintaining synchronized communication through backend APIs and shared database environments.

The machine learning modules responsible for graph processing and fraud prediction operate independently from the frontend monitoring dashboard. Processed fraud predictions, transaction classifications, and generated Suspicious Transaction Reports are synchronized through backend services to support real-time visualization and compliance management within the monitoring environment. This modular integration strategy simplifies scalability, maintenance, and future extension of the proposed framework.

4.2.3 Data Storage Management

The proposed TBML detection framework utilizes both relational and graph-based database systems to support efficient financial data management and analytical processing. PostgreSQL is utilized for storing structured transactional records, authentication information, compliance-related data, and generated Suspicious Transaction Reports. The relational database environment supports secure storage and efficient retrieval of operational monitoring information within the framework.

Memgraph is utilized as the graph database environment for storing interconnected financial entities and transactional relationships. Accounts, organizations, invoices, and transactions are represented as graph nodes and edges to support relational fraud analysis and graph neural network processing. The combined relational and graph-based storage strategy improves scalability, analytical flexibility, and efficient synchronization between backend analytical services and monitoring dashboards.

4.3 Overall System Architecture

The overall architecture of the proposed TBML detection framework is designed to support intelligent financial transaction monitoring, graph-based fraud analysis, federated learning, and automated compliance reporting within distributed institutional environments. The framework integrates transaction processing services, heterogeneous graph construction mechanisms, machine learning-based fraud detection modules, and real-time monitoring interfaces to identify suspicious transactional relationships associated with Trade-Based Money Laundering activities.

The architecture begins with multiple input sources including banking systems, trade systems, and external financial data providers. Raw transactional records, trade-related documents, KYC information, and institutional monitoring logs are initially processed within the transaction processing module. This stage performs data ingestion, validation, cleansing, normalization, and structured event generation to prepare the data for analytical processing.

The processed data is subsequently transformed into heterogeneous graph structures within the graph construction module. Financial entities such as accounts, organizations, invoices, and transactions are represented as interconnected nodes and edges to capture hidden financial dependencies and suspicious transactional patterns. The generated graph structures consist of node features, edge relationships, and connectivity

information required for graph-based machine learning operations.

The fraud detection layer incorporates a Hybrid Graph Neural Network branch for relational analysis and a temporal learning branch for capturing sequential transaction behavior. These analytical modules process graph structures and institutional transaction sequences to learn complex laundering patterns associated with suspicious activities. The outputs generated from these models are combined to improve fraud representation learning and transaction risk prediction accuracy.

To preserve institutional privacy and regulatory compliance, the framework integrates a federated learning environment in which participating institutions perform localized model training without directly sharing sensitive financial information. Local model updates generated within institutional environments are transmitted to the global aggregation module, where the updates are combined to construct an improved global fraud detection model while maintaining privacy-preserving collaborative learning.

The updated global model is utilized within the inference and risk scoring module to classify suspicious transactions and generate fraud risk scores. Transactions identified as potentially suspicious are forwarded to the compliance management environment for alert generation, Suspicious Transaction Report creation, and regulatory integration support. The overall architecture of the proposed TBML detection framework is illustrated in Figure **??**.

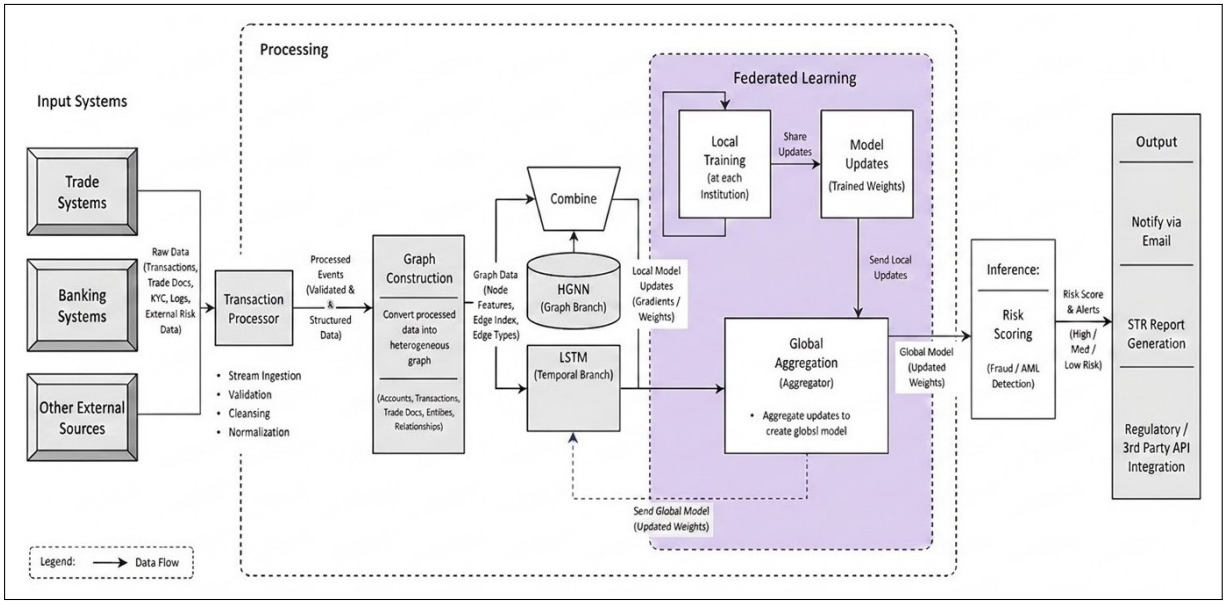

Figure 4.1: Overall Architecture of the Proposed TBML Detection Framework

4.4 Data Flow Diagrams

4.4.1 Data Flow Diagram Level 0

4.4.2 Data Flow Diagram Level 1

4.4.3 Data Flow Diagram Level 2

4.5 Summary

Chapter 5

Detailed Design of the Proposed TBML Framework

CHAPTER 5

DETAILED DESIGN OF THE PROPOSED TBML FRAMEWORK

The detailed design of the proposed TBML detection framework focuses on the internal organization, analytical workflow, and module-level interaction associated with graph-based fraud detection and real-time financial monitoring. The framework integrates transaction processing services, heterogeneous graph construction, hybrid HGNN–LSTM analytical models, federated learning mechanisms, and compliance reporting modules within a unified monitoring environment. This chapter presents the structure chart, detailed module design, database schema organization, and monitoring workflow associated with the proposed TBML detection framework.

5.1 Structure Chart

The structure chart of the proposed TBML detection framework illustrates the hierarchical organization of the major functional modules operating within the system. The framework is divided into frontend, backend, fraud detection, federated learning, database, and compliance layers to support scalable financial monitoring and intelligent fraud analysis within distributed Anti-Money Laundering environments.

The frontend layer provides dashboard interfaces, transaction monitoring services, risk visualization, and STR management functionalities for institutional users. Backend services coordinate API communication, transaction processing, authentication workflows, and access control operations between frontend interfaces and analytical modules. The fraud detection layer incorporates Hybrid HGNN–LSTM components responsible for graph-based analysis, temporal transaction modeling, feature fusion, and risk classification.

The federated learning layer enables privacy-preserving collaborative model training and global model synchronization between participating institutions. Database services manage transactional storage, graph relationships, audit logs, and compliance-related records, while the compliance layer supports STR generation, regulatory reporting, email notifications, and monitoring operations. The overall structural organization of the proposed TBML detection framework is illustrated in Figure **??**.

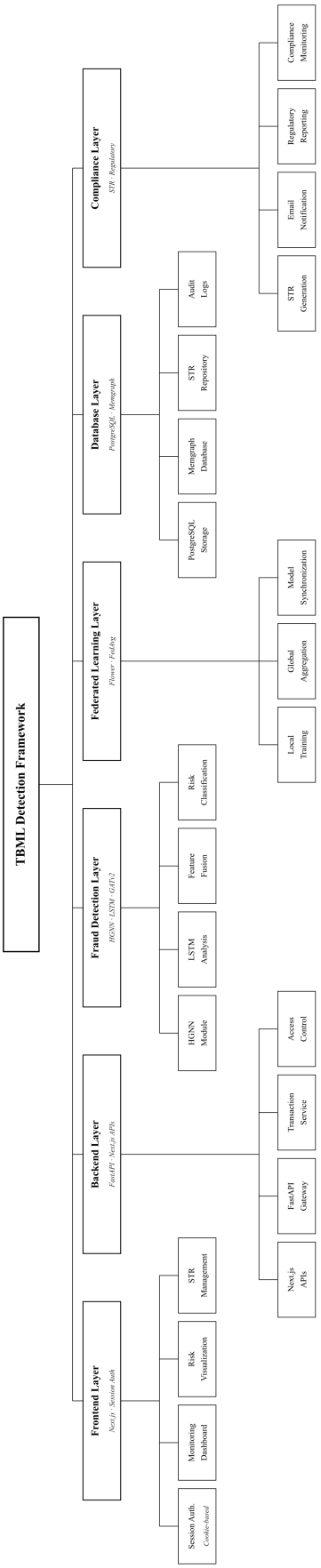

Figure 5.1: Structure Chart of the Proposed TBML Detection Framework

The structure chart illustrates the modular organization and interaction between analytical, monitoring, storage, and compliance-oriented services within the proposed framework. This modular design approach improves scalability, maintainability, service integration, and efficient coordination between frontend interfaces, backend analytical services, and distributed fraud detection environments.

5.2 Functional Description of the Modules

The functional description of the modules explains the internal processing workflow and interaction between the major analytical, monitoring, and compliance-oriented components operating within the proposed TBML detection framework. The framework integrates transaction streaming services, graph construction mechanisms, hybrid HGNN–LSTM analytical models, institutional verification environments, and regulatory reporting systems to support intelligent Trade-Based Money Laundering detection and real-time compliance monitoring.

5.2.1 Module Interaction Workflow

The module interaction workflow illustrates the coordinated operational flow between transaction ingestion services, streaming environments, graph-based analytical modules, fraud classification mechanisms, institutional verification procedures, and regulatory monitoring systems functioning within the proposed framework. Financial transactions, KYC records, and trade documents are initially processed within the transaction processing layer and published into Kafka streaming topics for asynchronous event-driven communication between backend analytical services.

Kafka consumer services subsequently consume the transaction streams and forward the processed data to the graph construction and feature extraction module where heterogeneous financial transaction graphs are generated using accounts, transactions, trade entities, and document relationships stored within the Memgraph database. The extracted graph embeddings and relational features are processed by the Hybrid HGNN–LSTM fraud detection module to perform graph representation learning, temporal transaction analysis, and intelligent fraud prediction.

The generated fraud classifications and monitoring results are stored within the PostgreSQL database and visualized through the dashboard monitoring environment for institutional investigators and banking administrators. Suspicious transactions identified

by the analytical system are reviewed by authorized bank administrators before being categorized as clean, suspicious, or confirmed fraudulent transactions.

Transactions verified as fraudulent are forwarded to the STR generation environment where Suspicious Transaction Reports are generated with analytical assistance from the Groq API service. Generated STR reports are subsequently reviewed by regulatory authorities responsible for investigation management, inquiry procedures, and enforcement operations. Notification management services coordinate fraud alerts and institutional communication processes, while the Resend email service delivers notification emails and regulatory enforcement messages to the corresponding institutional authorities. The overall module interaction workflow of the proposed TBML detection framework is illustrated in Figure **??**.

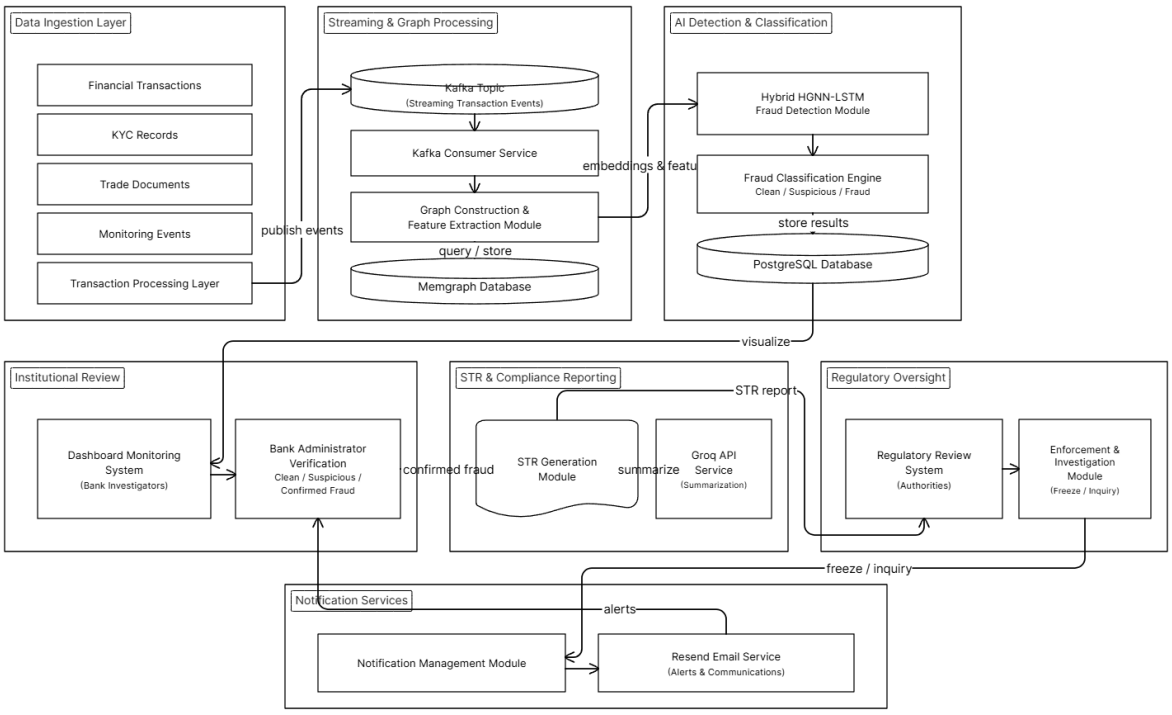

Figure 5.2: Workflow Interaction Diagram for the Proposed TBML Detection Framework

The workflow interaction diagram illustrates the coordinated interaction between streaming services, graph-based analytical modules, monitoring environments, institutional verification systems, and regulatory compliance services operating within the proposed framework. Such modular workflow integration supports scalable transaction processing, intelligent fraud analysis, compliance management, and real-time Anti-Money Laundering monitoring within distributed financial environments.

5.2.2 Transaction Processing Module

The transaction processing Module acts as the initial ingestion environment within the proposed TBML detection framework. This Module collects financial transactions, KYC records, trade documents, and institutional monitoring events originating from banking systems and external financial environments.

The input data processed within this Module includes sender and receiver account identifiers, transaction amounts, commodity types, trade quantities, trade values, invoice details, transaction timestamps, and institutional monitoring attributes associated with financial activities. The collected transactional data is validated, normalized, and converted into structured transaction records to support reliable downstream analytical processing.

The processed transaction events are published into Kafka streaming topics to enable asynchronous communication between backend services and analytical modules. The generated transaction streams serve as the primary input for graph construction, feature extraction, and graph-based fraud detection operations within the proposed framework.

5.2.3 Graph Construction Module

The graph construction module is responsible for transforming processed transaction streams into heterogeneous financial transaction graphs for graph-based analytical processing. This module receives validated transaction events from Kafka consumer services and models financial interactions between accounts, transactions, trade documents, and institutional entities within a connected graph environment.

The generated graph schema consists of interconnected node entities including `(:Account)`, `(:Transaction)`, and `(:Document)` connected through graph relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]`. These relationships represent sender-receiver interactions, transaction ownership, trade associations, and document linkages required for identifying hidden transactional dependencies associated with Trade-Based Money Laundering activities.

The module interacts with the Memgraph database to store graph structures, entity relationships, and transactional connectivity information required for downstream analytical learning. The generated graph features and relational representations are subsequently forwarded to the Hybrid HGNN–LSTM fraud detection module for graph-based fraud analysis and intelligent transaction classification.

5.2.4 Hybrid HGNN–LSTM Fraud Detection Module

The Hybrid HGNN–LSTM fraud detection module performs graph-based fraud analysis and temporal transaction modeling within the proposed TBML detection framework. The module receives KYC node features, transaction sequences, SWIFT edge attributes, and trade metadata generated from the graph construction environment for downstream analytical processing.

The graph branch utilizes multi-layer GATv2 operations to perform neighborhood aggregation and graph attention learning across interconnected financial entities. In parallel, the temporal branch utilizes LSTM-based sequential modeling to analyze transaction sequences and identify abnormal temporal transaction behavior associated with suspicious financial activities. The trade branch processes trade metadata and institutional transaction attributes using multilayer perceptron operations and feature normalization mechanisms.

The generated graph embeddings, temporal representations, and trade feature vectors are combined within the feature fusion layer for downstream fraud prediction and transaction risk analysis. The dense classification layer categorizes transactions into predefined classes including `CLEAN`, `WATCHLIST`, and `FRAUD`. The overall architecture of the proposed Hybrid HGNN–LSTM fraud detection module is illustrated in Figure **??**.

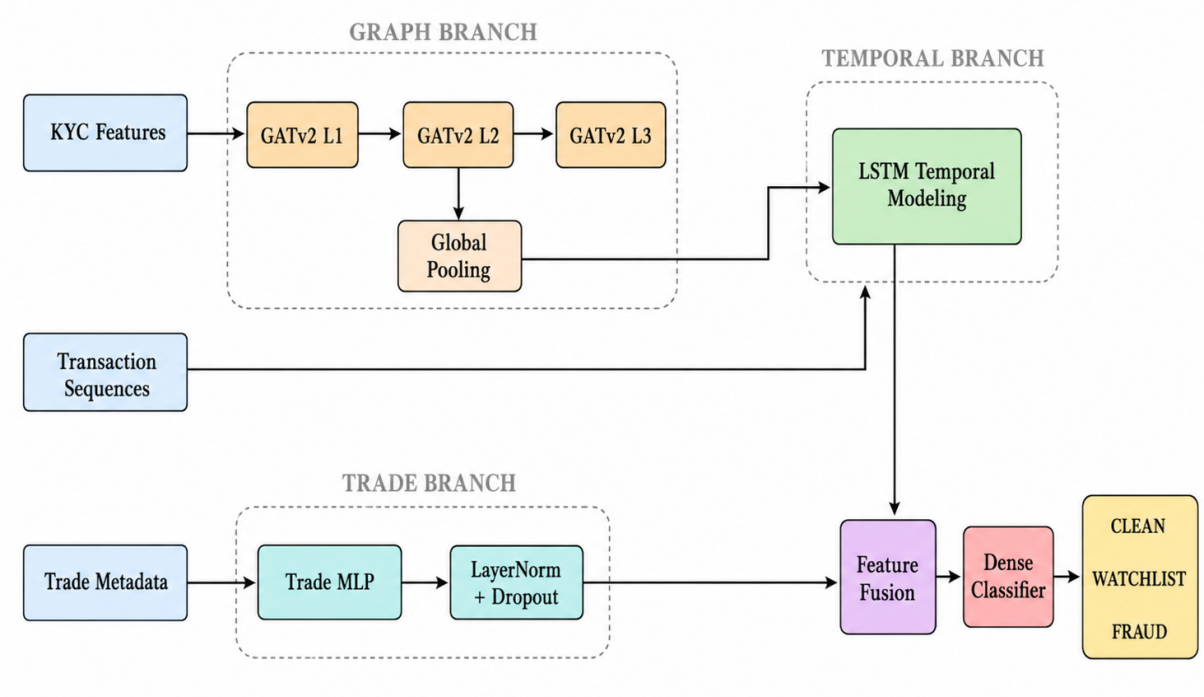

Figure 5.3: Architecture of the Proposed Hybrid HGNN–LSTM Fraud Detection Module

5.2.5 Federated Learning Module

The federated learning module enables privacy-preserving collaborative fraud detection across distributed financial institutions without directly sharing sensitive transactional data. Each participating institution performs local model training using institutional transaction graphs, transaction sequences, and fraud classification labels generated within the analytical environment.

The locally trained model parameters are periodically transmitted to the central aggregation server where the Federated Averaging (`FedAvg`) strategy combines local model updates to generate an optimized global fraud detection model. The aggregated global model is subsequently redistributed to participating institutions for continuous fraud detection and real-time analytical inference. The overall federated learning architecture of the proposed framework is illustrated in Figure **??**.

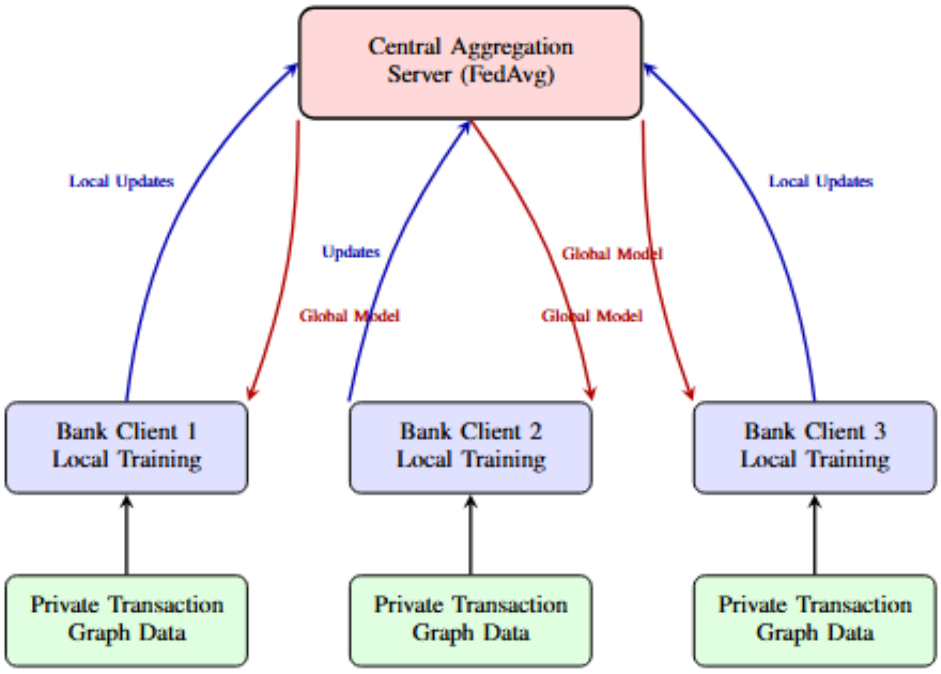

Figure 5.4: Federated learning architecture for privacy-preserving collaborative TBML detection across distributed institutions

STR Generation and Notification Module

The STR generation and notification module is responsible for compliance reporting and fraud alert management within the proposed TBML detection framework. Transactions verified as fraudulent by authorized banking administrators are forwarded to this

module for Suspicious Transaction Report (STR) generation and regulatory processing. The module generates STR reports containing transaction details, fraud classifications, trade metadata, institutional information, and investigation summaries associated with suspicious financial activities.

The Groq API service is utilized for analytical summarization and automated assistance during compliance documentation generation. Generated STR reports are subsequently forwarded to regulatory monitoring authorities for investigation and enforcement procedures. The notification management environment coordinates fraud alerts and institutional communication workflows, while the Resend email service delivers notification emails, investigation alerts, and regulatory enforcement messages to banking institutions and regulatory authorities.

Dashboard and Regulatory Monitoring Module

The dashboard and regulatory monitoring module provides real-time transaction visualization, fraud monitoring, and institutional investigation functionalities within the proposed TBML detection framework. The monitoring dashboard displays transaction classifications, fraud alerts, risk scores, STR records, and analytical monitoring information generated by the fraud detection environment. Separate dashboard environments are maintained for banking administrators and regulatory authorities to support institution-level monitoring and centralized compliance management operations.

Authorized banking administrators can view suspicious and fraudulent transactions identified by the analytical model and perform institutional verification procedures before categorizing transactions as `CLEAN`, `WATCHLIST`, or `FRAUD`. Suspicious transactions can either be escalated as fraudulent transactions or marked as clean after institutional verification procedures, thereby reducing false positive classifications within the monitoring environment. Transactions verified as fraudulent are subsequently forwarded for Suspicious Transaction Report (STR) generation and regulatory reporting.

Regulatory authorities monitor generated STR reports and institution-level fraud cases through centralized monitoring dashboards. The regulatory environment supports compliance monitoring, document verification requests, inquiry procedures, and account freeze operations associated with suspicious financial activities.

5.3 Summary

This chapter presented the detailed design of the proposed TBML detection framework including the structural organization, module interaction workflow, functional description of analytical modules, federated learning architecture, and database management environment associated with the system. The chapter discussed the interaction between transaction processing services, graph construction mechanisms, hybrid HGNN–LSTM analytical modules, dashboard monitoring environments, and compliance reporting systems operating within the proposed framework.

The detailed design further illustrated how streaming-based transaction processing, graph-based fraud analysis, federated collaborative learning, institutional verification, STR generation, and regulatory monitoring are integrated within a unified Anti-Money Laundering environment. The subsequent chapter discusses the implementation details, backend integration and dashboard development associated with the proposed TBML detection framework.

Chapter 6

Implementation

CHAPTER 6

IMPLEMENTATION

Every chapter should start with an introduction paragraph. This paragraph should brief about the flow of the chapter. This introduction can be limited within 4 to 5 sentences. The chapter heading should be appropriately modified (a sample heading is shown for this chapter). But don't start the introduction paragraph in the chapters like "This chapter deals with....". Instead you should bring in the highlights of the chapter in the introduction paragraph.

6.1 Conclusion

This chapter should not contain an introduction paragraph like other chapters. You can directly write conclusion of the work done under this section. Typically this section can have 3 to 4 paragraphs.

First paragraph should bring in the scenario of the project and every objective should be explained here.

Second paragraph should say how the objectives are implemented and achieved.

Last paragraph should draw the conclusions from each objective with quantitative results, performance improvement etc.

6.2 Future Scope

Briefly discuss the constraints and limitations of the project and state the possibilities of extending the work in future.

6.3 Learning Outcomes of the Project

- List the learning outcomes here
- List a minimum of 5 learning outcomes

Chapter 7

Results & Discussions

CHAPTER 7

RESULTS & DISCUSSIONS

Every chapter should start with an introduction paragraph. This paragraph should brief about the flow of the chapter. This introduction can be limited within 4 to 5 sentences. The chapter heading should be appropriately modified (a sample heading is shown for this chapter).But don't start the introduction paragraph in the chapters like "This chapter deals with....". Instead you should bring in the highlights of the chapter in the introduction paragraph.

7.1 Contents of this chapter

All the results obtained for your objectives should be discussed in this chapter. This chapter should contain the following sections as per the project.

1. Simulation results
2. Experimental results
3. Performance Comparison
4. Inferences drawn from the results obtained

All the figures should be properly explained by bringing the scenarios of the design done in the project. A detailed discussion of results obtained should be done in this chapter.

7.2 Tables in thesis

- All Table Caption should be in Sentence Case, TNR 10 Pt. It should be of the Format:
  - Table 1.1 Results of the experiment ....(Centered)
- It should be cited as Table 1.1.
- Caption should appear above the Table.
- Table Header and the entries should be of Font TNR 10 Pt, Justified.
- For wider Table, the page orientation can be Landscape.

- For Larger Table, it can run to pages and the header should be repeated for each page of the Table.
- Table must be adjusted to fit in the page and no single row is left out for a new page.

Sample Table **??** is given below for your reference,

Table 7.1: Country List

Country Name or Area Name	**ISO ALPHA 2 Code**	**ISO ALPHA 3 Code**	**ISO numeric Code**
Afghanistan	AF	AFG	004
Aland Islands	AX	ALA	248
Albania	AL	ALB	008
Algeria	DZ	DZA	012
American Samoa	AS	ASM	016
Andorra	AD	AND	020
Angola	AO	AGO	024

7.3 Math equation in thesis

All equation should be written using equation editor or using an equivalent tool.

- Equations should be numbered as : 1.1, 1.2 ...
- Equation should be Centered, 12 Pt, TNR.
- Equation number should be right Justified
- It should be cited as Eqn. 1.1.
- If the sentence starts by citing an equation, then it should be written as Equation 1.1 For example, Equation 5.1 states the Pythagoras theorem.

For example in Eqn. **??**, The well known Pythagorean theorem $x^2 + y^2 = z^2$ was proved to be invalid for other exponents. Meaning the next equation has no integer solutions:

$$x^n + y^n = z^n \tag{7.1}$$

The mass-energy equivalence is described by the famous equation in Eqn. **??**

$$E = mc^2 \tag{7.2}$$

discovered in 1905 by Albert Einstein.

7.4 Interpretation of results

The chapters should not end with figures, instead bring the paragraph explaining about the figure at the end followed by a summary paragraph.

After elaborating the various sections of the chapter (From Chapter 2 onwards), a summary paragraph should be written discussing the highlights of that particular chapter. This summary paragraph should not be numbered separately. This paragraph should connect the present chapter to the next chapter.

Chapter 8

Conclusion and Future Scope

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

Every chapter should start with an introduction paragraph. This paragraph should brief about the flow of the chapter. This introduction can be limited within 4 to 5 sentences. The chapter heading should be appropriately modified (a sample heading is shown for this chapter). But don't start the introduction paragraph in the chapters like "This chapter deals with....". Instead you should bring in the highlights of the chapter in the introduction paragraph.

8.1 Conclusion

This chapter should not contain an introduction paragraph like other chapters. You can directly write conclusion of the work done under this section. Typically this section can have 3 to 4 paragraphs.

First paragraph should bring in the scenario of the project and every objective should be explained here.

Second paragraph should say how the objectives are implemented and achieved.

Last paragraph should draw the conclusions from each objective with quantitative results, performance improvement etc.

8.2 Future Scope

Briefly discuss the constraints and limitations of the project and state the possibilities of extending the work in future.

8.3 Learning Outcomes of the Project

- List the learning outcomes here
- List a minimum of 5 learning outcomes

Appendix A

Code

APPENDIX A

CODE

A.1 First Appendix

You can use `tcblisting` for creating the code snippets. The following example illustrates how one can customize the `tcblisting` to achieve the `tcl` script. Similarly, one can use it for other programming language listing, including HDL.

```
# Since our design has a clock with name clk,
## specify that name under [get_port ]
create_clock -period 40 -waveform {0 20} [get_ports clk]

# Setting a 'delay' on the clock:
set_clock_latency 0.3 clk

# Setting up constraints on your I/P and O/P pins
set_input_delay 2.0 -clock clk [all_inputs]
set_output_delay 1.65 -clock clk [all_outputs]

# Set realistic 'loads' on each output pin
set_load 0.1 [all_outputs]

# Set 'maximum' fanin and fan-out for the input and output pins
set_max_fanout 1 [all_inputs]
set_fanout_load 8 [all_outputs]
```

BIBLIOGRAPHY

[1] IMTF Compliance Technologies, *When trade becomes a cover: Unpacking trade-based money laundering*, 2024. [Online]. Available: https://imtf.com/blog/when-trade-becomes-a-cover-unpacking-trade-based-money-laundering/

[2] Financial Action Task Force and Egmont Group, "Trade-based money laundering: Trends and developments," FATF, Paris, France, Tech. Rep., 2020. [Online]. Available: https://fatf-gafi.org

[3] United Nations Office on Drugs and Crime, *Money laundering and Illicit financial flows overview*, 2024. [Online]. Available: https://unodc.org

[4] Europol, *Criminal finances, money laundering and asset recovery*, 2024. [Online]. Available: https://europa.eu

[5] J. Saenz and J. Lewer, "Quantifying illicit financial flows and trade misinvoicing in developing economies," *Journal of Financial Crime Research*, vol. 18, no. 2, pp. 112–128, 2024.

[6] R. Z. Z. L. W. Y. D. N. K.-Y. L. Jiani Fan Lwin Khin Shar, "Deep learning approaches for anti-money laundering on mobile transactions: Review, framework, and directions," *IEEE Internet of Things Journal*, vol. 13, pp. 10 407–10 428, 2026, ISSN: 2327-4662. DOI: 10.1109/JIOT.2026.3653434

[7] A. I. Rasmus Ingemann Tuffveson Jensen, "Fighting money laundering with statistics and machine learning," *IEEE Access*, vol. 11, pp. 8889–8903, 2023, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2023.3239549

[8] J. P. Y. G. G. Y. Jie Song Sijia Zhang, "Dimension expansion for learning money laundering activities hidden in transaction network," *IEEE Transactions on Computational Social Systems*, vol. 13, pp. 251–262, 2026, ISSN: 2329-924X. DOI: 10.1109/TCSS.2025.3599534

[9] P. Z. J. P. Y. G. G. Y. Jie Song Sijia Zhang, "Illicit social accounts? anti-money laundering for transactional blockchains," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 391–404, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2024.3518068

[10] C. J. Zhong Li Xueting Yang, "Multi-view graph-based hierarchical representation learning for money laundering group detection," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2035–2050, 2025, ISSN: 1556-6021. DOI: 10.1109/TIFS.2025.3529321

[11] E. S. Abdul Haluk Batur Gezici, "Pagerank-based unsupervised deep vertex representations for anti-money laundering detection," *IEEE Access*, vol. 13, pp. 196 935–196 950, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3634197

[12] S. A. C. P. D. P. A. M. Md. Rezaul Karim Felix Hermsen, "Scalable semi-supervised graph learning techniques for anti money laundering," *IEEE Access*, vol. 12, pp. 50 012–50 029, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3383784

[13] B. Y. Kyungmo Koo Minyoung Park, "A suspicious financial transaction detection model using autoencoder and risk-based approach," *IEEE Access*, vol. 12, pp. 68 926–68 939, 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3399824